

Advance java Day Wise

Day 1:

What is JDBC?

Key Features:

- **Platform-independent** (thanks to Java)

JDBC (Java Database Connectivity) is an API provided by the `java.sql` module (introduced in Java 9's module system). It allows Java applications to connect to and interact with relational databases.

- **Partially DB-independent** (due to SQL syntax variations between databases)
-

JDBC-related Packages

1. `java.sql` – Standard DB connectivity (basic JDBC features)
 2. `javax.sql` – Advanced features (e.g., connection pooling)
-

Why JDBC?

- Enables Java apps to communicate with databases
- Supports **WORA (Write Once, Run Anywhere)**
- Promotes **loose coupling** using interfaces

DB Independence is Partial Because:

SQL syntax can vary across databases, requiring changes in:

- Queries (especially in DAO layer)
 - JDBC drivers
-

DB-Specific Configurations Required in JDBC:

1. **JDBC Driver JAR** (e.g., `mysql-connector.jar`)
 2. **Database URL**
 3. **Username**
 4. **Password**
 5. **SQL queries (may vary by DB)**
-

JDBC Architecture

- **JDBC API** provides interfaces (`Connection` , `Statement` , `ResultSet` , etc.)
 - **JDBC Driver** (in JAR form) provides DB-specific implementations
 - Drivers convert Java API calls ↔ DB calls
-

JDBC Driver Types (You should refer to slides for diagrams, but here's a quick list):

1. **Type 1:** JDBC-ODBC Bridge
 2. **Type 2:** Native-API Driver
 3. **Type 3:** Network Protocol Driver
 4. **Type 4:** Thin Driver (Pure Java, widely used today)
-

Generic Steps in JDBC Programming

1. Load JDBC Driver (auto in JDBC 4.0+)
2. Establish Connection:

```
Connection con = DriverManager.getConnection(url, user, password);
```

3. Create Statement/PreparedStatement
 4. Execute SQL query
 5. Process ResultSet
 6. Close connection and other resources
-

Examples

- **Login without Layers:**

```
SELECT * FROM users WHERE email = ? AND password = ?
```

- **Using Layers:**

- **Tester**
 - **DAO Interface & Implementation**
 - **DBUtils (for connection logic)**
 - **POJO (User)**
 - **RDBMS (e.g., MySQL)**
-

CRUD Objectives

1. Display all user details
 2. Login validation
 3. Filter users born between dates
 4. Register new customer
 5. Update password
 6. Delete customer
-

✓ Why JDBC?

JDBC provides **database independence** by allowing Java applications to interact with any RDBMS using a **standard API**, while the implementation is handled by **DB-specific drivers**.

- **API Layer:** Java provides `java.sql` interfaces (e.g., `Connection`, `Statement`).
- **Driver Layer:** DB vendors implement these interfaces (e.g., `com.mysql.cj.jdbc.Driver` for MySQL).

✓ Steps for JDBC Connectivity:

1. Add JDBC Driver to Classpath

- JAR file (e.g., `mysql-connector-java-8.0.xx.jar`).

2. Load & Register Driver

- `Class.forName("com.mysql.cj.jdbc.Driver");`

3. Establish Connection

- `DriverManager.getConnection(url, user, pass);`

4. Create Statement / PreparedStatement / CallableStatement

- For general queries: `Statement`
- For parameterized queries: `PreparedStatement`
- For stored procedures/functions: `CallableStatement`

5. Execute Query

- `executeQuery()` → for SELECT (returns `ResultSet`)
- `executeUpdate()` → for DML/DDDL (returns row count)
- `execute()` → for stored procedures

6. Process ResultSet

- Use `rs.next()` to iterate
- Use `getXXX()` methods to extract column data

7. Close Resources

- Always close `ResultSet`, `Statement`, and `Connection` (preferably in `finally` or use try-with-resources).

✓ PreparedStatement vs Statement:

Feature	Statement	PreparedStatement
Precompiled	✗	✓ (Faster for repeated execution)
Supports IN parameters	✗	✓ (e.g., <code>pst.setInt(1, 101);</code>)
SQL Injection safe	✗	✓

✓ CallableStatement:

Used to **call stored procedures/functions** and handle IN, OUT, and INOUT parameters.

Example:

```
CallableStatement cst = cn.prepareCall("{call proc_name(?, ?)}");
cst.setInt(1, val);
cst.registerOutParameter(2, Types.INTEGER);
cst.execute();
int out = cst.getInt(2);
```

✅ Transaction Management:

To group multiple SQL operations into a single **atomic unit**:

```
cn.setAutoCommit(false);
try {
    // execute multiple queries
    cn.commit();
} catch(SQLException e) {
    cn.rollback();
}
```

✅ Scrollable / Updatable ResultSet:

```
Statement stmt = cn.createStatement(
    ResultSet.TYPE_SCROLL_SENSITIVE,
    ResultSet.CONCUR_UPDATABLE
);
ResultSet rs = stmt.executeQuery("SELECT * FROM table");
```

Use `.absolute(n)`, `.updateXXX()`, `.updateRow()`, `.insertRow()`, `.deleteRow()` for modifications.

✅ Metadata in JDBC:

- `DatabaseMetaData` — info about DB, tables, driver
- `ResultSetMetaData` — info about columns in ResultSet

✅ Handling BLOBs (Binary Large Objects):

- For insert: `pst.setBinaryStream(...)`
- For read: `Blob b = rs.getBlob(); b.getBytes(...)`

✅ Complete DB Independence:

Use a `.properties` file to store DB config (driver, URL, username, password). Load with:

```
Properties props = new Properties();
props.load(new FileReader("dbconfig.properties"));
```

Day 2:

How to Protect a JDBC Application Against SQL Injection

About SQL Injection

SQL Injection is one of the **OWASP Top 10** most critical security vulnerabilities. It allows attackers to **inject malicious SQL queries** through application input forms to gain unauthorized access to database information.

What Makes Applications Vulnerable?

When developers construct SQL queries by **directly appending user input** into a SQL string, attackers can modify the input to change the behavior of the query.

Vulnerable Example:

```
String sql = "SELECT * FROM users WHERE user_name = '" + uname + "' AND password = '" + pass + "'";  
Statement st = cn.createStatement();  
st.executeQuery(sql);
```

If a user inputs something like:

```
uname: a  
pass: b' OR '1'='1
```

The query becomes:

```
SELECT * FROM users WHERE user_name = 'a' AND password = 'b' OR '1'='1';
```

This condition always evaluates to true, potentially bypassing authentication entirely.

How to Prevent SQL Injection

1. Use **PreparedStatement** Instead of **Statement**

- Prepared statements **separate SQL logic from data**.
- SQL statements are **precompiled**, and user inputs are treated as **data**, not part of the SQL command.
- Prevents the attack by **escaping dangerous characters** like quotes.

Safe Example with PreparedStatement:

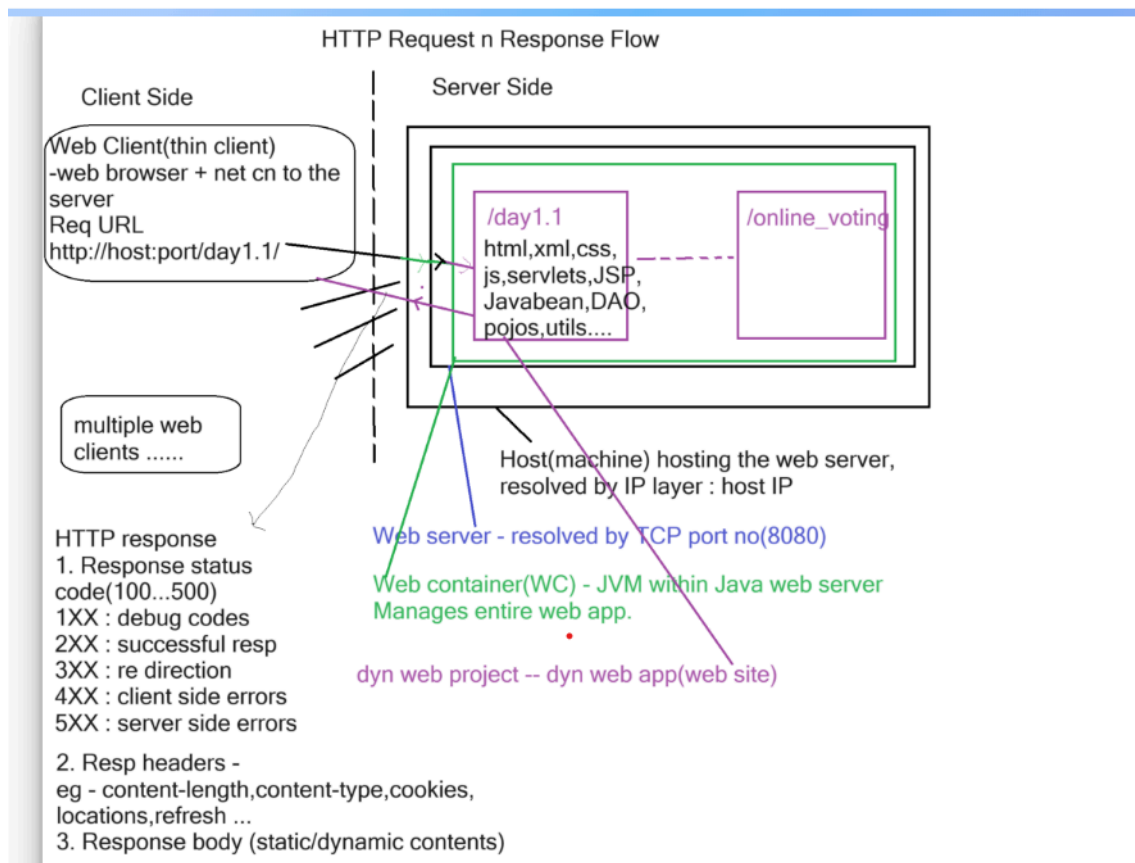
```
String sql = "SELECT * FROM users WHERE user_name = ? AND password = ?";  
PreparedStatement ps = cn.prepareStatement(sql);  
ps.setString(1, uname);  
ps.setString(2, pass);  
ResultSet rs = ps.executeQuery();
```

🔒 Why This Works:

- `setString()` and other `setType()` methods **bind the value securely**, so even if a user inputs a malicious string, it's not executed as part of the SQL command.
- The database sees the query structure and the data **separately**, making injections impossible.

🧱 Other Best Practices:

- **Input Validation:** Ensure only expected formats are accepted (e.g., regex for usernames).
- **Least Privilege Principle:** Use database accounts with limited access rights.
- **Stored Procedures (with care):** Preferably those that do not dynamically build queries.
- **ORM Frameworks:** Consider using Hibernate, JPA, etc., which abstract away SQL.
- **Use Web Security Tools:** Periodically scan your app with tools like OWASP ZAP or SQLMap.



📍 Overview: Client-Server Web Architecture

◆ Client Side

- **Web Client (Thin Client):** Usually a web browser.
- **Sends HTTP Request:** Uses a URL like `http://host:port/day1.1/` to send a request to the server.

Server Side Components

1. Host (Machine):

- Identified via **IP address** (resolved by IP layer).
- Hosts the **web server**.

2. Web Server (e.g., Apache Tomcat):

- Resolved via **TCP port** (commonly **8080** for development).
- Receives incoming HTTP requests.
- Delegates request to the **Web Container** (Servlet Container).

3. Web Container (WC):

- A part of the Java web server.
- Runs inside a **JVM**.
- Manages:
 - Servlets
 - JSP
 - JavaBeans
 - DAO
 - Utils
 - Sessions
 - Lifecycle of the entire **web application**

4. Web Applications (Dynamic Projects):

- Like `/day1.1/` or `/online_voting/`, deployed inside the container.
 - Contain: `.jsp`, `.html`, `.css`, `.js`, Java classes, config files.
-

HTTP Request & Response

◆ HTTP Request:

- Sent by browser with:
 - Method (GET/POST)
 - URL
 - Headers
 - Cookies
 - Body (for POST)

◆ HTTP Response:

1. **Status Code** (100–500 range):

- **1XX** : Informational
- **2XX** : Success
- **3XX** : Redirection
- **4XX** : Client errors
- **5XX** : Server errors

2. Headers:

- **Content-Type** , **Content-Length** , **Set-Cookie** , etc.

3. Body:

- Static or dynamic content generated by server (HTML, JSON, etc.)



HttpSession and Cookies



When a user connects:

1. Server checks if a **JSESSIONID** **cookie** is present.
2. If not present:
 - Server creates a new **HttpSession** object.
 - Sends back a **Set-Cookie: JSESSIONID=xyz** header in the HTTP response.
3. On subsequent requests:
 - Browser sends the **JSESSIONID** in the **cookie header**.
 - Server uses it to **retrieve the session** object and maintain user state (e.g., login info, shopping cart).



Summary

- **Web clients** make requests to **web apps** hosted on a **Java web server**.
- Web container handles servlet and JSP execution.
- Sessions are managed using **HttpSession** and **cookies**.
- Responses include **status codes**, **headers**, and **body** data.

Day 3:



Database Transactions in JDBC



What is a Transaction?

- A **logical group of SQL statements** representing a business operation.
- **Atomicity**: All operations must succeed together or fail entirely.
- Changes are **permanent only if all succeed**.



Example: Product Purchase Transaction

1. Check product availability
2. Validate & update customer's credit/debit limit
3. Update stock

If any of these steps fail, the entire transaction must be rolled back.

How to Handle Transactions in JDBC

1. Start a Transaction

Disable auto-commit (default is true).

```
Connection cn = DriverManager.getConnection(...);
cn.setAutoCommit(false); // Start transaction
```

2. Wrap DB Logic in try-catch Block

```
try {
    // Business logic with multiple SQL statements
    // Step 1: Check product
    // Step 2: Update credit
    // Step 3: Update stock

    cn.commit(); // 3. Commit if all succeed
} catch (SQLException e) {
    cn.rollback(); // 4. Rollback entire transaction if any failure
} finally {
    cn.setAutoCommit(true); // 5. Reset auto-commit to default
}
```

Partial Rollback with Savepoint

6. Set Savepoint

```
Savepoint sp1 = cn.setSavepoint();
```

7. Rollback to Savepoint

```
cn.rollback(sp1); // Roll back only to this point
```

Key JDBC APIs for Transactions

Action	JDBC API
Start transaction	<code>conn.setAutoCommit(false)</code>

Commit transaction	<code>conn.commit()</code>
Rollback all	<code>conn.rollback()</code>
Create Savepoint	<code>Savepoint sp = conn.setSavepoint();</code>
Rollback to point	<code>conn.rollback(sp);</code>
Resume auto-commit	<code>conn.setAutoCommit(true)</code>

What is `CallableStatement` ?

In **JDBC**, `CallableStatement` is an interface used to **call stored procedures and functions** defined in your database. These are like pre-written SQL blocks stored on the database server.

Real-Life Analogy

Imagine a **restaurant kitchen** (the database). Instead of telling the chef every step for cooking a dish (writing SQL every time), you just **place an order** (call a stored procedure), like:

| "Give me a burger with cheese and no onions."

Here, you're calling a predefined recipe (stored procedure) and **passing parameters** like "with cheese" or "no onions".

Why Use It?

- Avoid rewriting complex SQL repeatedly.
- Easy maintenance — logic is centralized in the DB.
- Can return **multiple OUT values**.
- Performance: Stored procedures are precompiled.
- Better security (you only call, not modify structure).

Step-by-Step JDBC Example

Stored Procedure in MySQL

```
DELIMITER //

CREATE PROCEDURE get_discounted_price(
    IN original_price DOUBLE,
    IN discount_percent DOUBLE,
    OUT final_price DOUBLE
)
BEGIN
    SET final_price = original_price - (original_price * discount_percent / 100);
END //
```

DELIMITER ;

This procedure takes:

- `original_price` (IN)
- `discount_percent` (IN)
- Returns `final_price` (OUT)

✓ Java JDBC Code Using `CallableStatement`

```
import java.sql.*;

public class DiscountCalculator {
    public static void main(String[] args) {
        try {
            Connection conn = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/your_db", "root", "password");
            CallableStatement cst = conn.prepareCall("{call get_discounted_price(?, ?, ?)}");
        } {
            // Step 1: Set IN parameters
            cst.setDouble(1, 1000.0); // original price
            cst.setDouble(2, 10.0);   // discount percent

            // Step 2: Register OUT parameter
            cst.registerOutParameter(3, Types.DOUBLE);

            // Step 3: Execute
            cst.execute();

            // Step 4: Get the result from OUT parameter
            double discountedPrice = cst.getDouble(3);
            System.out.println("Discounted Price: " + discountedPrice);

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Notes

- **IN parameters** → You set values using `.setType()`.
- **OUT parameters** → You must register them using `.registerOutParameter()`.
- You can have **INOUT** parameters by doing both.
- Return values from **stored functions** use the `?=call funcName(...)` format.

Tip for Interviews

Question: How do you handle INOUT parameters?

Answer: I set their value with `setType()` and also register their type using `registerOutParameter()` before executing the statement.

What is a Servlet?

A **Servlet** is a **Java class** used to **build dynamic web applications**. It runs on a **Java-enabled web server** (like Tomcat) and handles **HTTP requests and responses**.

Think of it as a **Java program that listens to browser requests** and **sends back responses** (like HTML, JSON, etc).

Why is `HttpServlet` declared as abstract, even though it has 100% concrete methods?

Full Explanation:

1. It's abstract to enforce a design pattern

Even though all methods in `HttpServlet` have real (concrete) implementations, it's marked `abstract` because **it's not meant to be used directly**.

2. To force you to override key methods

The most important methods (`doGet()` , `doPost()` , etc.) have default implementations that return:

HTTP 405 - Method Not Allowed

Which is useless unless overridden.

So, it's **mandatory for developers to override** at least one of them (`doGet()` , `doPost()` , etc.) depending on your application's need.

3. Why not just make the methods abstract instead of the whole class?

Because:

- Some default behavior (e.g., `service()` method) is reusable and implemented concretely.
 - You don't have to override **every** method — just the one(s) you need (e.g., only `doPost()`).
 - Making the whole class `abstract` prevents someone from **accidentally instantiating a servlet** without overriding anything — which would be a programming error.
-

Summary

Feature	Purpose
<code>HttpServlet</code> is <code>abstract</code>	Prevents direct instantiation of a useless object
Default methods exist	So you only override what you need (e.g., <code>doGet</code> , <code>doPost</code>)

No abstract methods in class	So subclassing is optional unless behavior is needed
Prevents programming error	Ensures developer must override relevant HTTP method(s)

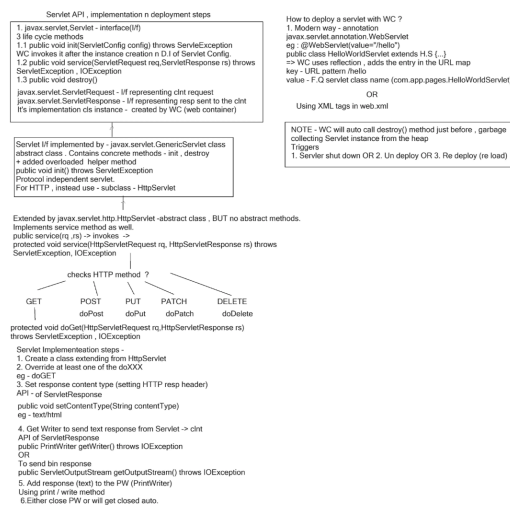
✓ Real-world Analogy

Think of `HttpServlet` like a **template form**:

It provides full structure but says:

“You must fill in at least one section (like `doGet`) before using this!”

Otherwise, it's a **blank, non-functional form**.



🔄 Servlet Life Cycle (Servlet API Cycle)

The **Servlet API cycle** defines how the servlet is loaded, initialized, serves requests, and is destroyed.

It consists of **three main phases**:

1. Loading and Instantiation

- The servlet container (like Tomcat) **loads the servlet class** when the application starts or on the first request (depending on configuration).
- It **creates only one object** of the servlet (singleton).

🔧 2. Initialization (`init()`)

- Called **once** after the servlet is instantiated.
- The `init(ServletConfig config)` method is called to allow the servlet to initialize resources (e.g., DB connection, file reading, etc.).

```
@Override
public void init() throws ServletException {
```

```
// Initialization code here
}
```

3. Request Handling (`service()`)

- Every time a request comes, the `service()` method is called by the container.
- It determines the HTTP method (GET, POST, etc.) and **dispatches** it to the correct handler method (`doGet()` , `doPost()` , etc.).

```
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
    // Handle GET request
}
```

Note:

You generally override `doGet()` or `doPost()` rather than `service()` itself.

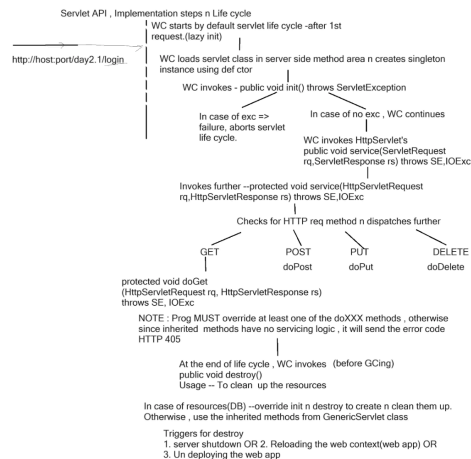
4. Destruction (`destroy()`)

- Called **once** when the servlet is being removed (e.g., on server shutdown or redeployment).
- Use this to release resources like DB connections, threads, etc.

```
@Override
public void destroy() {
    // Cleanup code
}
```

Servlet Lifecycle Summary

Phase	Method	Called When
Instantiation	Constructor	When the servlet is first loaded
Initialization	<code>init()</code>	After instantiation (only once)
Request Handling	<code>service()</code> → <code>doGet()</code> / <code>doPost()</code>	Every request
Destruction	<code>destroy()</code>	When servlet is being unloaded



Page Navigation = Moving the user from one page to another.

1. Client Pull (Redirect)

- User is taken to the next page in the *next* request.
- New URL is generated by the browser.

1.1. Manual Redirect (User Clicks)

Example: Clicking a link or button.

1.2. Automatic Redirect (sendRedirect)

- Server tells browser to go to another URL.
- Browser issues a **new GET request** to that URL.

Code:

```
response.sendRedirect("admin_page");
```

What Happens Internally:

- Response buffer is **discarded**.
- Servlet container sends:
 - Status: **302 (Temporary Redirect)**
 - Header: `Location: admin_page`
- Browser issues **new request**:

```
GET /context_path/admin_page
```

Caveat:

- If response is already committed (buffer flushed/closed), `sendRedirect()` throws `IllegalStateException`.

✓ 2. Server Pull (Forward / Include)

- **Same request** is used to navigate between resources.
- Also called **Request Chaining** or **Resource Dispatching**.

Step 1: Create RequestDispatcher

```
RequestDispatcher rd = request.getRequestDispatcher("next_page");
```

2.1. 🔄 Forward (Internal Navigation)

- Transfers control to another resource on the **server-side**.
- No new request from client.

✓ Code:

```
rd.forward(request, response);
```

🔍 Details:

- Response buffer cleared before forwarding.
- Only the *last* resource generates output.
- Used for **division of responsibility** among servlets/JSPs.

⚠️ Caveat:

- Throws `IllegalStateException` if response already committed.

2.2. + Include (Server-side include)

- Includes output of another resource **within** current response.

✓ Code:

```
rd.include(request, response);
```

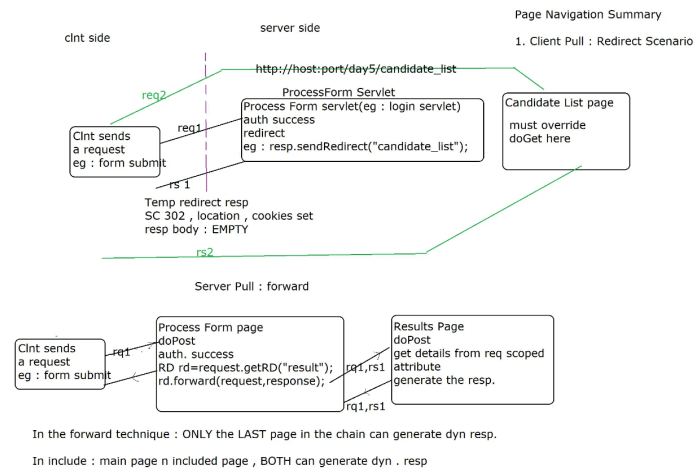
🔍 Details:

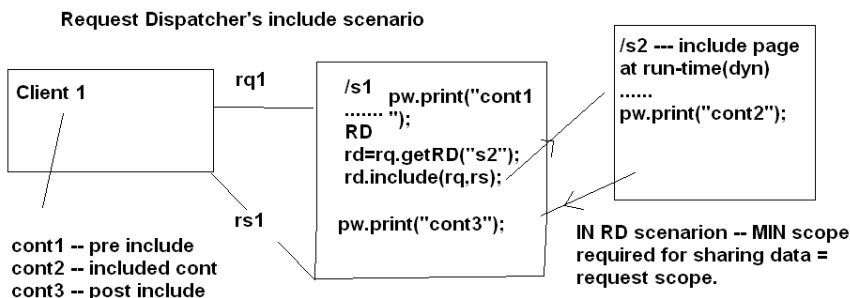
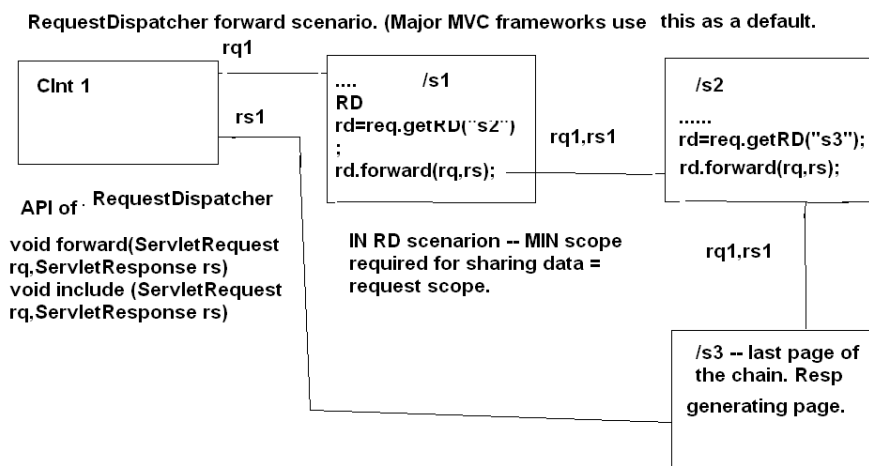
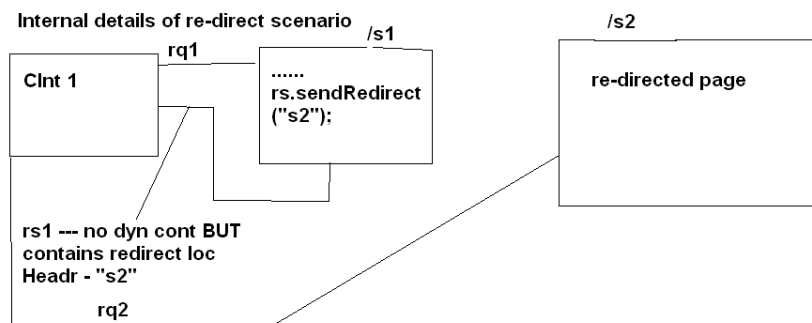
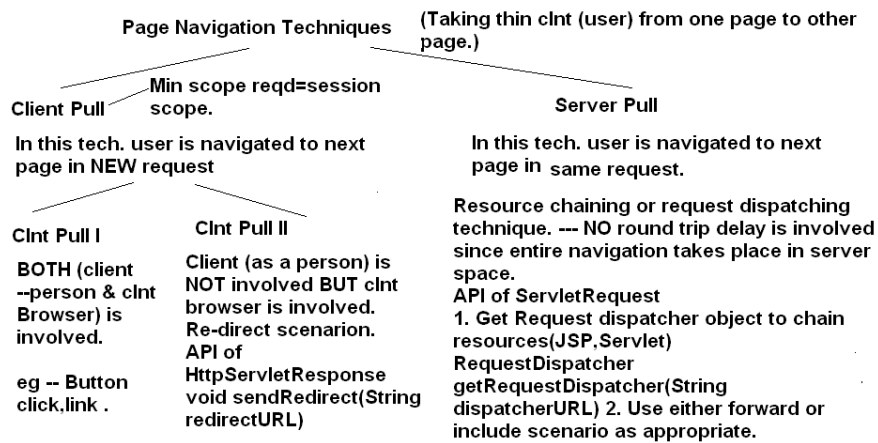
- Used for reusing content (e.g., header/footer).
- Included resource **cannot** set response headers or status code.

Summary Table:

Feature	Client Pull (Redirect)	Server Pull (Forward)	Server Pull (Include)
Request Type	New	Same	Same

URL in Browser	Changes	Same	Same
Used For	Navigation	Division of logic	Reusability
Headers/Status	Can change	Last page only	Ignored in include
Exception Risk	If response committed	If response committed	No such issue





Day 4:



What is a Session?

A **session** represents a **conversational state** between a **client and server** over multiple HTTP requests/responses.

Since HTTP is **stateless**, sessions are used to maintain **identity** and **state** across requests (e.g., from login to logout).

Why Do We Need State Management?

1.  To **identify specific clients** among multiple.
2.  To **remember data** across multiple requests (e.g., shopping cart, user details).

 **Example:** A shopping cart must know which user's cart to add items to, even across multiple pages.

Session Duration

- Typically: From **login to logout** or until **timeout**.
-  **Default session timeout** in Tomcat: **30 minutes**

Server-Side State Management Techniques in Java EE

1. Plain Cookie-Based Scenario

What is a Cookie?

- Small **text data**, created by server, stored in client's browser.
- Shared across multiple dynamic pages for same client.

Cookie Lifecycle:

1. **Create Cookie:**

```
Cookie c = new Cookie("user", "Akshay");
```

2. **Send Cookie to Client:**

```
response.addCookie(c);
```

3. **Retrieve on Next Request:**

```
Cookie[] cookies = request.getCookies();
```

4. **Control Expiry:**

```
c.setMaxAge(3600); // 1 hour
```

Disadvantages:

- Developer manually manages cookies.

- Only **text** data is supported.
 - **Increased traffic** with more cookies.
 - Cookies are **client-side**, so disabled cookies → session tracking fails.
-

2. HttpSession Interface-Based Tracking

In this approach, session **data is stored server-side**, while a **session ID cookie** is managed by the **Web Container**.

Steps:

1. **Get/Create Session:**

```
HttpSession session = request.getSession(); // or getSession(true/false)
```

2. **Store Attributes:**

```
session.setAttribute("cart", cartList);
```

3. **Retrieve Attributes:**

```
List<CartItem> cart = (List<CartItem>) session.getAttribute("cart");
```

4. **Remove Attribute:**

```
session.removeAttribute("cart");
```

5. **Invalidate Session** (e.g., logout):

```
session.invalidate();
```

6. **Check if Session is New:**

```
session.isNew();
```

7. **Set Session Timeout (Programmatically):**

```
session.setMaxInactiveInterval(300); // 5 minutes
```

8. **Set Timeout in `web.xml` :**

```
<session-config>  
  <session-timeout>10</session-timeout> <!-- in minutes →  
</session-config>
```

9. **Get Session ID:**

```
session.getId();
```

10. List All Attributes:

```
Enumeration<String> names = session.getAttributeNames();
```

✅ Advantages Over Plain Cookies:

- Server manages object storage.
- Can store **Java objects**.
- Reduced network traffic (only session ID travels).
- No need to manually handle cookies.

❌ Limitation:

- If cookies are disabled, session tracking fails **unless using URL rewriting**.

3. 🧵 HttpSession + URL Rewriting (Alternative)

- Embeds **session ID** in the **URL** instead of using cookies.
- Useful when cookies are **disabled** on the client.

Example:

```
String encodedURL = response.encodeURL("dashboard.jsp");
```

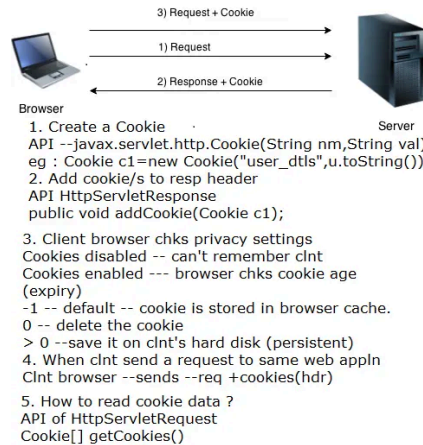
📦 What is an Attribute?

An attribute is a **key-value pair stored server-side**, created by the developer:

```
session.setAttribute("user", new User());
```

🧭 Scopes of Attributes:

Scope	Visibility
Request	For current request only
Session	For the current session (same client)
Application	Shared across all requests in the same app



Day 6:

◆ Why Executor Framework?

- **Problem with Threads:** Creating a new thread per task = overhead (CPU, memory).
- **Solution:** Reuse a fixed number of threads to handle many tasks = better scalability, performance, and control.

◆ Core Interfaces & Classes

Component	Purpose
<code>Executor</code>	Basic interface for executing <code>Runnable</code> tasks via <code>execute()</code> method.
<code>ExecutorService</code>	Sub-interface of <code>Executor</code> for managing lifecycle and task execution (<code>submit</code> , <code>shutdown</code> , etc.).
<code>ScheduledExecutorService</code>	Schedules tasks to run after delay or periodically.
<code>Executors</code> (utility class)	Factory for creating preconfigured thread pools.
<code>Callable<V></code>	Represents a task that returns a result and may throw an exception.
<code>Future<V></code>	Represents the result of an async computation.

◆ Executors Factory Methods

Method	Description
<code>newFixedThreadPool(n)</code>	Creates thread pool with fixed size <code>n</code> .
<code>newCachedThreadPool()</code>	Unbounded pool, reuses threads, kills idle threads.
<code>newSingleThreadExecutor()</code>	One thread only (like a queue).
<code>newScheduledThreadPool(n)</code>	Fixed size pool that can schedule tasks with delays or periodically.

◆ Steps to Use Executor Framework with Runnable

```
ExecutorService executor = Executors.newFixedThreadPool(10);
```

```

Runnable task = () → {
    System.out.println("Task executed by " + Thread.currentThread().getName());
};

executor.execute(task);
executor.shutdown();

```

◆ Steps with Callable & Future

```

ExecutorService executor = Executors.newFixedThreadPool(3);

Callable<Integer> task = () → {
    Thread.sleep(1000);
    return 42;
};

Future<Integer> future = executor.submit(task);

try {
    Integer result = future.get(); // Blocks until result is ready
    System.out.println("Result: " + result);
} catch (InterruptedException | ExecutionException e) {
    e.printStackTrace();
}

executor.shutdown();

```

◆ Common Methods of **ExecutorService**

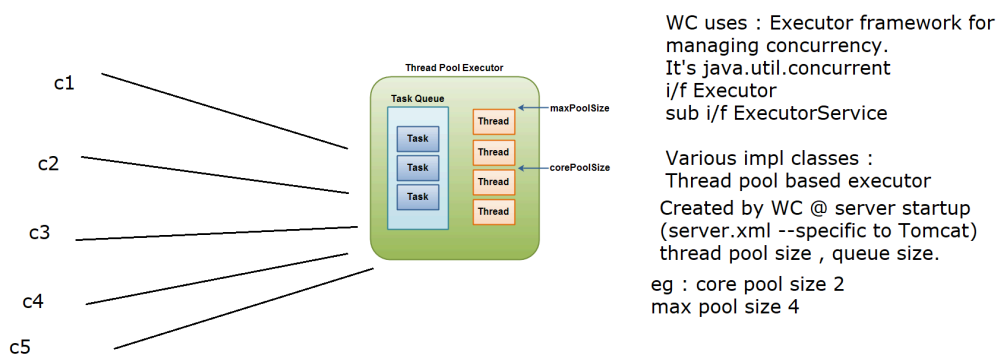
Method	Description
<code>execute(Runnable)</code>	Executes a <code>Runnable</code> task.
<code>submit(Callable)</code>	Submits task and returns <code>Future</code> .
<code>invokeAll(Collection<Callable>)</code>	Blocks until all tasks complete.
<code>shutdown()</code>	Initiates graceful shutdown.
<code>shutdownNow()</code>	Attempts to stop all tasks and returns those not yet executed.
<code>awaitTermination(timeout, unit)</code>	Blocks until termination or timeout.

◆ Callable vs Runnable

Feature	Runnable	Callable
Returns a result?	✗	✓
Throws checked exceptions?	✗	✓
Functional interface?	✓ (<code>run()</code>)	✓ (<code>call()</code>)

Would you like a real-world use case or advanced features like custom thread pools,

`CompletionService` , or `ForkJoinPool` ?



🔄 HTTP Session Working (Step-by-Step Flow)

📌 Step 1: Client Sends Initial Request (No Session Yet)

- A **new user** opens your website and makes a request to the server.
- Example: `GET /login.jsp`

📌 Step 2: Server Creates a New Session

- The server (Servlet or JSP) calls:

```
HttpSession session = request.getSession();
```

- Since it's a new user, the Web Container:
 - **Creates a new `HttpSession` object** in server memory.
 - **Generates a unique session ID (`JSESSIONID`)** for this client.
 - Sends it to the client in the **Set-Cookie header** of the response.

```
Set-Cookie: JSESSIONID=ABC123XYZ; Path=/; HttpOnly
```

📌 Step 3: Browser Stores Session ID

- The browser saves the `JSESSIONID` cookie.
- This ID will be sent back in **every future request** to the same web app.

```
Cookie: JSESSIONID=ABC123XYZ
```

📌 Step 4: Server Receives Subsequent Requests

- Browser automatically includes the `JSESSIONID` cookie in all future requests.
- The server reads this cookie and:
 - Maps it to the existing `HttpSession` object in its memory.
 - Allows retrieval of previously stored attributes.

Step 5: Server Maintains Session Data

- The developer can store objects using:

```
session.setAttribute("user", new User("Akshay"));
```

- These attributes remain available throughout the session:

```
User user = (User) session.getAttribute("user");
```

Step 6: Session Timeout or Logout

- If there is **no activity** for a certain time (default = 30 mins in Tomcat), the session expires.
- Developer can also **manually invalidate** session:

```
session.invalidate();
```

- After this, a **new session** is created on the next request.

Key Points:

Feature	Detail
Session ID	Sent via cookie (<code>JSESSIONID</code>)
Session Storage	Server-side (<code>HttpSession</code> object in memory)
Data Type Supported	Any Java object (Serializable recommended)
Session Timeout	Default 30 min (customizable)
Fails if Cookies Off	Use URL rewriting as fallback

Optional Fallback: URL Rewriting

If cookies are disabled, use:

```
response.encodeURL("dashboard.jsp");
```

This appends the session ID to the URL:

```
dashboard.jsp;jsessionid=ABC123XYZ
```


- You can create attributes that are shared across the entire application (global scope) using `ServletContext`. These attributes are thread-safe, but always access them in a thread-safe manner (e.g., using synchronized blocks).

```
getServletContext().setAttribute("appData", someObject);
```

5.3 Accessing Global Parameters:

- **Context Parameters** in `web.xml` are available to all servlets in the web application.

Adding Parameters in `web.xml` :

```
<context-param>
  <param-name>user_name</param-name>
  <param-value>abc</param-value>
</context-param>
```

- **Accessing Parameters in a Servlet:**
- You can access these parameters from **any servlet** in the app after it's initialized.

```
String userName = getServletContext().getInitParameter("user_name");
```

5.4 Creating Request Dispatcher:

- You can create a **RequestDispatcher** to forward requests or include content in a servlet or JSP.

```
RequestDispatcher dispatcher = getServletContext().getRequestDispatcher("/somePage.jsp");
```

Additional `ServletContext` Methods:

1. Get ServletContext for the Web Application:

- Can be accessed via the `getServletContext()` method in a servlet.

```
ServletContext context = getServletContext();
```

2. Get Resource (e.g., file in the web app):

- You can retrieve resources like files inside your web application directory.

```
URL resource = getServletContext().getResource("/images/logo.png");
```

3. Set/Get Application Attributes:

- Set attributes that will be shared across all servlets in the web application.

```
getServletContext().setAttribute("appData", appData);
```

- Get the value of attributes.

```
Object appData = getServletContext().getAttribute("appData");
```

4. Get Context Path:

- Get the context path of the web application (e.g., `/myapp`).

```
String contextPath = getServletContext().getContextPath();
```

Thread Safety and Synchronization:

- **Attributes** stored in `ServletContext` are **global** (application-level), and you should handle access to them in a **thread-safe manner**. This means you should use synchronization mechanisms when accessing shared resources to avoid race conditions.

```
synchronized (getServletContext()) {  
    // Access shared context data  
}
```

Example: Accessing Context Parameter and Logging

In a servlet, to access a context parameter and log it:

```
@WebServlet("/example")  
public class ExampleServlet extends HttpServlet {  
    @Override  
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {  
        String username = getServletContext().getInitParameter("user_name");  
        getServletContext().log("Retrieved user_name: " + username);  
  
        response.getWriter().write("Hello, " + username);  
    }  
}
```

Flow of ServletContext Creation:

1. Web Application is deployed.
2. **ServletContext** is created by the Web Container.
3. Servlet can retrieve the **ServletContext** and access context parameters, attributes, and other resources.

javax.servlet.ServletConfig Overview

1. What is `ServletConfig` ?

- `ServletConfig` is a servlet-specific configuration object.

- It is created by the **Web Container (WC)** to pass initialization parameters specific to the servlet during its **initialization** phase.
- These parameters are used to configure the servlet instance when it is created.

2. Who Creates Its Instance?

- The **Web Container (WC)** creates an instance of `ServletConfig` for each servlet when the servlet is initialized.

3. When Is It Created?

- After the Web Container creates the **servlet instance** (using the default constructor), it creates the `ServletConfig` object.
- The `ServletConfig` is then passed to the servlet's `init()` method, which is invoked during servlet initialization.
- This is an example of **Dependency Injection (DI)**, where the WC injects the necessary configuration information into the servlet.

4. How Many Instances of `ServletConfig` Exist?

- There is **one `ServletConfig` instance per servlet**. Each servlet can have its own configuration parameters, which are only accessible to that specific servlet.

5. Usage of `ServletConfig` :

- **Store Servlet-Specific Initialization Parameters:**
 - These parameters are specific to the servlet and cannot be accessed by other servlets.
 - They are often used to configure servlet-specific settings during initialization.



Where to Add Servlet-Specific Init Parameters?

Option 1: In `web.xml` Configuration File

In the `web.xml` file, you can add servlet-specific initialization parameters within the `<servlet>` tag:

```
<servlet>
  <servlet-name>init</servlet-name>
  <servlet-class>com.example.MyServlet</servlet-class>
  <init-param>
    <param-name>name</param-name>
    <param-value>value</param-value>
  </init-param>
</servlet>

<servlet-mapping>
  <servlet-name>init</servlet-name>
  <url-pattern>/test_init</url-pattern>
</servlet-mapping>
```

Option 2: Using `@WebServlet` Annotation

You can also add servlet-specific init parameters using the `@WebServlet` annotation:

```
@WebServlet(value = "/test",
    initParams = {
        @WebInitParam(name = "nm1", value = "val1"),
        @WebInitParam(name = "nm2", value = "val2")
    })
public class MyServlet extends HttpServlet {
    // Servlet code
}
```

How to Access Servlet-Specific Init Params?

Step-by-Step Guide:

Step 1: Override the `init()` Method

- The `init()` method is called during servlet initialization. You can access the `ServletConfig` object inside this method.

```
@Override
public void init() throws ServletException {
    // Accessing the init parameters here
}
```

Step 2: Get the `ServletConfig`

- Inside the `init()` method, you can retrieve the `ServletConfig` object using the `getServletConfig()` method.

```
ServletConfig config = getServletConfig();
```

Step 3: Access Init Parameters

- You can access the servlet-specific initialization parameters using the `getInitParameter(String paramName)` method.

```
String paramValue = config.getInitParameter("name");
```

Example:

Here's an example of a servlet that uses `ServletConfig` to retrieve initialization parameters:

```
@WebServlet(value = "/test",
    initParams = {
        @WebInitParam(name = "username", value = "admin"),
        @WebInitParam(name = "password", value = "password123")
    })
```

```

    })

    public class MyServlet extends HttpServlet {

        @Override
        public void init() throws ServletException {
            // Access the ServletConfig object
            ServletConfig config = getServletConfig();

            // Retrieve initialization parameters
            String username = config.getInitParameter("username");
            String password = config.getInitParameter("password");

            // Log or use the parameters
            System.out.println("Username: " + username);
            System.out.println("Password: " + password);
        }

        @Override
        protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
            // Servlet logic here
        }
    }
}

```

In this example:

- The servlet retrieves the **username** and **password** initialization parameters using `ServletConfig`.
- The parameters are provided in the `@WebServlet` annotation.

Key Points About `ServletConfig` :

Feature	Description
Who Creates It	Web Container (WC)
When Is It Created	After the servlet instance is created, before <code>init()</code> method is called
Where to Add Parameters	<code>web.xml</code> or <code>@WebServlet</code> annotation
Usage	Stores servlet-specific initialization parameters
Access to Parameters	Use <code>getServletConfig().getInitParameter("paramName")</code>

What is a Servlet Listener (or Web Application Listener)?

A **Servlet Listener** is an interface in the Servlet API that allows you to listen to events during the lifecycle of a web application. These events can involve actions such as:

- The creation or destruction of requests.
- The creation or destruction of sessions.
- The creation or destruction of the web application context.
- Changes to request, session, or context attributes.

Servlet listeners are used to handle such events and take action accordingly, making them useful for logging, resource management, or any other cross-cutting concerns that need to be triggered by specific events.

Listener Interfaces in Servlet API:

There are several key listener interfaces in the Servlet API. Here are a few commonly used ones:

1. `ServletRequestListener`

- This listener allows you to react to the lifecycle of a `ServletRequest` object, specifically when the request is created or destroyed.

- **Methods:**

- `void requestDestroyed(ServletRequestEvent sre)` - Invoked when the request is destroyed.
- `void requestInitialized(ServletRequestEvent sre)` - Invoked when the request is initialized.

2. `HttpSessionListener`

- This listener allows you to react to the lifecycle of an HTTP session, specifically when a session is created or destroyed.

- **Methods:**

- `void sessionCreated(HttpSessionEvent se)` - Invoked when an HTTP session is created.
- `void sessionDestroyed(HttpSessionEvent se)` - Invoked when an HTTP session is destroyed.

3. `ServletContextListener`

- This listener allows you to react to the lifecycle of the **web application context**, which is essentially the entire web application. You can respond when the web application is initialized or destroyed.

- **Methods:**

- `void contextDestroyed(ServletContextEvent sce)` - Invoked when the context (web app) is destroyed.
- `void contextInitialized(ServletContextEvent sce)` - Invoked when the context (web app) is initialized.

4. Other EventListener Interfaces:

- There are other specific listener interfaces that can be used depending on the event type, such as `ServletContextAttributeListener` or `HttpSessionAttributeListener`, which react to the addition, removal, or modification of context or session attributes.

Event Handling Steps:

To use listeners, follow these steps:

1. Create a Separate Class Implementing the Listener Interface:

- Implement the required listener interface in your class.
- For example, if you're interested in session events, implement `HttpSessionListener`.

2. Register the Listener with the Web Container:

- **Option 1: Using the `@WebListener` Annotation:**

- If you're using Servlet 3.0+ (which supports annotations), you can use the `@WebListener` annotation to mark the listener class.

```
@WebListener
public class MyHttpSessionListener implements HttpSessionListener {
    @Override
    public void sessionCreated(HttpSessionEvent se) {
        // Code to handle session creation
    }

    @Override
    public void sessionDestroyed(HttpSessionEvent se) {
        // Code to handle session destruction
    }
}
```

- **Option 2: Registering the Listener in `web.xml` :**

- If you're using an older version of the Servlet API or prefer XML configuration, you can register the listener in the `web.xml` configuration file.

```
<listener>
  <listener-class>com.example.MyHttpSessionListener</listener-class>
</listener>
```

Example: Using a `ServletContextListener`

A `ServletContextListener` can be used to handle events that occur when the web application is started or shut down.

```
@WebListener
public class MyServletContextListener implements ServletContextListener {
    @Override
    public void contextInitialized(ServletContextEvent sce) {
        // Code to initialize resources (e.g., database connections)
        System.out.println("Web Application Initialized");
    }

    @Override
    public void contextDestroyed(ServletContextEvent sce) {
        // Code to clean up resources (e.g., closing database connections)
        System.out.println("Web Application Destroyed");
    }
}
```

In this example:

- The `contextInitialized` method will be called when the web application is initialized.

- The `contextDestroyed` method will be called when the web application is destroyed.

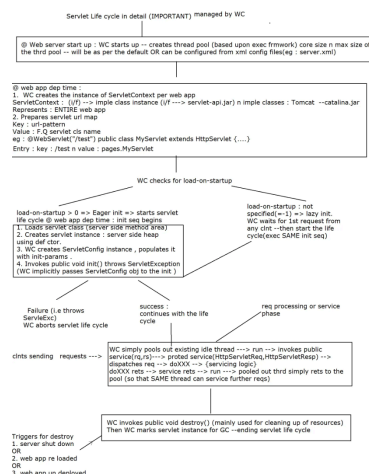
This can be helpful for performing one-time setup tasks or cleanup operations.

Common Use Cases for Listeners:

- **Logging:** You can log events like session creation or request destruction to track user activity.
- **Session Management:** For tracking user login times or handling session timeouts.
- **Resource Cleanup:** Closing database connections or releasing resources when a session or context is destroyed.
- **Global Initialization:** Loading global configuration or initializing services when the web application starts.

Summary of Key Listener Interfaces:

Listener Interface	Event Triggered	Methods
<code>ServletRequestListener</code>	When a request is created or destroyed	<code>requestInitialized(ServletRequestEvent)</code> <code>requestDestroyed(ServletRequestEvent)</code>
<code>HttpSessionListener</code>	When an HTTP session is created or destroyed	<code>sessionCreated(HttpSessionEvent)</code> <code>sessionDestroyed(HttpSessionEvent)</code>
<code>ServletContextListener</code>	When the web application context is created or destroyed	<code>contextInitialized(ServletContextEvent)</code> <code>contextDestroyed(ServletContextEvent)</code>
<code>ServletContextAttributeListener</code>	When context attributes are added, removed, or modified	<code>attributeAdded(ServletContextAttributeEvent)</code> <code>attributeRemoved(ServletContextAttributeEvent)</code> <code>attributeReplaced(ServletContextAttributeEvent)</code>



Day 7:

✓ 1. JSP Syntax Overview

JSP (Java Server Pages) allows you to write **HTML** combined with **Java code**. The Java code is executed on the server, and the result is sent as HTML to the browser.

- It's basically a `.jsp` file containing:
 - HTML code
 - JSP Tags or Directives
 - Java code (in scriptlets, expressions, or declarations)

✓ 2. JSP Lifecycle

The JSP lifecycle includes the following steps:

Phase	Description
Translation	JSP is converted into a Java Servlet (one-time process)
Compilation	The Servlet is compiled into a <code>.class</code> file
Instantiation	Servlet object is created
Initialization	<code>jspInit()</code> is called (once)
Request Processing	<code>jspService()</code> is called for every client request
Destruction	<code>jspDestroy()</code> is called before removing from memory

👉 This lifecycle ensures that your JSP page acts just like a servlet behind the scenes.

✓ 3. JSP Directives

JSP Directives provide global info about the page. They begin with `<%@` and are not directly displayed.

Directive	Syntax	Purpose
page	<code><%@ page ... %></code>	Sets properties like <code>language</code> , <code>import</code> , <code>session</code> , etc.
include	<code><%@ include file="..." %></code>	Includes a static file at translation time
taglib	<code><%@ taglib uri="..." prefix="..." %></code>	Enables use of custom tags like JSTL

Example:

```
<%@ page import="java.util.*, java.sql.*" %>
```

✓ 4. JSP Scripting Elements

Used to write Java code directly inside HTML.

Element	Syntax	Description
Scriptlet	<code><% code %></code>	General Java code (loops, if-else)
Expression	<code><%= expression %></code>	Outputs the result directly into HTML
Declaration	<code><%! declaration %></code>	Declares variables or methods at class level

Example:

```
<% int a = 5; %>
<%= "The value of a is " + a %>
<%! int square(int x) { return x*x; } %>
```

✓ 5. Expression Language (EL)

Used to access data like request attributes, session variables, JavaBeans, etc., in a **simple and clean** way — no need to write Java code.

Basic Syntax: `${expression}`

Example	What it does
<code>\${param.name}</code>	Gets value of input field named "name"
<code>\${sessionScope.user}</code>	Accesses session-scoped object "user"
<code>\${employee.salary}</code>	Accesses salary property of an <code>employee</code> bean

✓ 6. JavaBeans in JSP

JavaBeans are reusable Java classes that follow specific conventions (with private properties + public getters/setters). JSP uses them to separate business logic.

Steps:

1. Create a JavaBean class:

```
public class Student {
    private String name;
    public void setName(String name) { this.name = name; }
    public String getName() { return name; }
}
```

1. Use in JSP:

```
<jsp:useBean id="stu" class="com.akshay.Student" scope="session" />
<jsp:setProperty name="stu" property="name" value="Akshay" />
<jsp:getProperty name="stu" property="name" />
```

This displays: `Akshay`

✓ Goal

Develop a **web application independent of cookies** for session tracking.

✓ Why URL Rewriting?

When cookies are **disabled**, we still need to track sessions.

The **only info** the server needs from the client is the **Session ID** (usually `JSESSIONID`).

If it's not coming via cookie, then embed it directly in the URL.

✓ What is URL Rewriting?

Embedding the JSESSIONID into the URL itself to maintain session without cookies.

- This allows state management even when the browser doesn't support or allow cookies.
- Common places to embed session ID:
 - `href` links
 - `form` actions
 - `sendRedirect` URLs

✓ How does it work internally?

1. **Web Container (WC)** checks:

- First, if `JSESSIONID` is in a cookie.
- If not, it checks the **URL** for an encoded session ID.

2. If found, the server **extracts the session ID** and continues session tracking normally.

✓ API Methods for URL Rewriting

Situation	API Method	Purpose
Client-pull I (clicking links/buttons)	<code>encodeURL(String url)</code>	Encodes normal URLs
Client-pull II (sendRedirect)	<code>encodeRedirectURL(String url)</code>	Encodes redirect URLs

Behavior:

- If cookies are blocked:
 - returns URL with `;jsessionid=xyz123abc`
- If cookies are allowed:
 - returns plain URL (session ID already handled via cookie)

```
// Example
String safeURL = response.encodeURL("profile.jsp");
out.print("<a href='" + safeURL + "'>My Profile</a>");
```

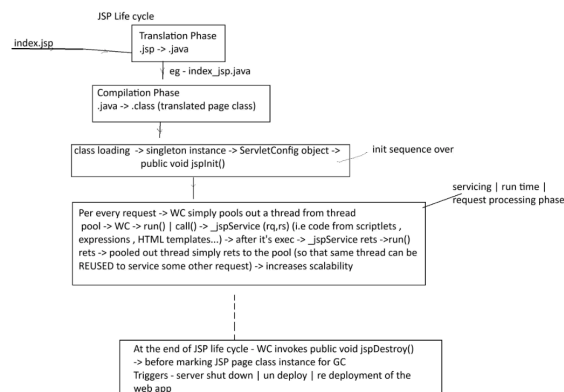
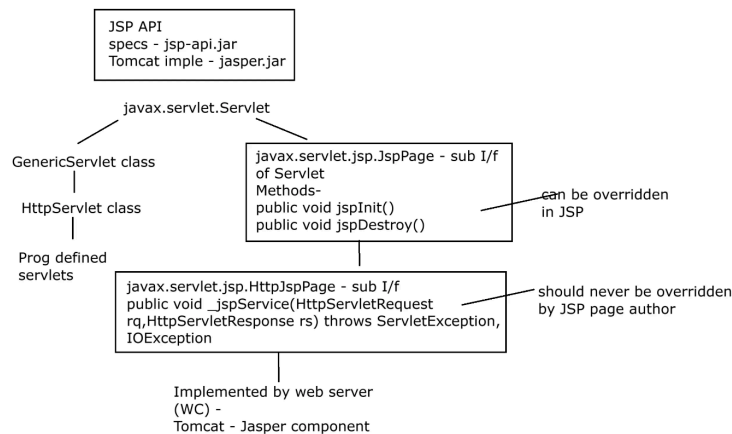
```
// sendRedirect example
String redir = response.encodeRedirectURL("dashboard.jsp");
response.sendRedirect(redir);
```

✗ Disadvantage

- **Not secure by default**
- Vulnerable to **Session Fixation attacks** (attacker provides a known session ID)

✓ Security Best Practice

- Always use **HTTPS** when URL rewriting is used
- Optionally:
 - Invalidate old session and generate a **new session ID** after login (`session.invalidate()` → `request.getSession(true)`)



🌐 JSP Overview

Covers:

- Why JSP?
- JSP Life-cycle
- JSP 2.x Syntax

📄 Template Data (Static Content)

- **HTML** or static content sent to client as-is.
- **Comments:**

- Server-side: `<%-- comment --%>` (Not sent to browser)
- Client-side: `<!-- comment -->` (Visible in browser source)

JSP Elements (Adds Dynamic Nature)

◆ Implicit Objects (Pre-configured Variables)

Accessible **only to scriptlets and expressions** (`<% ... %>` and `<%= ... %>`)

- `request` , `response` , `out` , `session` , `config` , `application` , `pageContext` , `page` , `exception`

◆ Scripting Elements

1. **Scriptlets:** `<% ... %>`
 2. **Expressions:** `<%= ... %>`
 3. **Declarations:** `<%! ... %>`
- **Expression Language (EL)** `${ }` is a modern alternative to scriptlets.

◆ EL Implicit Objects

Accessible **only in EL** (`${ }`):

- `param` , `pageScope` , `requestScope` , `sessionScope` , `applicationScope` , `cookie` , `initParam` , `pageContext`

◆ JSP Directives

Used to control the overall structure of the JSP page.

- `page` , `include` , `taglib`

Example:

```
<%@ page contentType="text/html" %>
<%@ include file="header.jsp" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
```

◆ JSP Actions

Standard Actions:

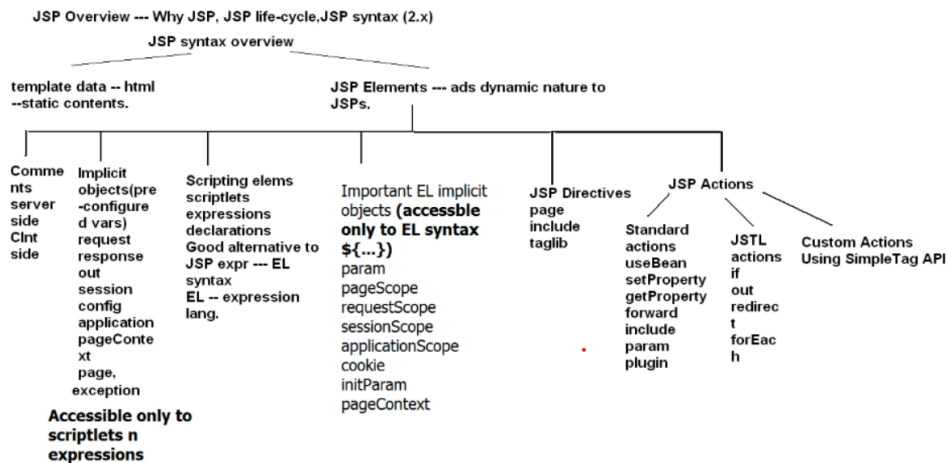
- `<jsp:useBean>` , `<jsp:setProperty>` , `<jsp:getProperty>`
- `<jsp:forward>` , `<jsp:include>` , `<jsp:param>` , `<jsp:plugin>`

JSTL (JSP Standard Tag Library) Actions:

- `<c:if>` , `<c:out>` , `<c:redirect>` , `<c:forEach>`

Custom Actions (via SimpleTag API):

- Create custom reusable tags
-



Day 8

✓ Summary of Key Concepts:

📌 What is JSP?

- JSP is a dynamic web technology used to generate HTML pages dynamically.
- Allows embedding Java code in HTML using special tags.
- Lifecycle managed by the **Web Container** (e.g., Tomcat).

📌 Why JSP?

1. Separation of **Presentation Logic (PL)** and **Business Logic (BL)**.
2. Auto-translation into servlets simplifies dynamic page creation.
3. JSPs are not commonly used in modern full-stack apps—replaced by frontend frameworks like **React/Angular/Vue**.

🌀 JSP Lifecycle:

1. Client request triggers JSP lifecycle.
2. **Translation:** `.jsp` → `.java` (servlet).
3. **Compilation:** `.java` → `.class`.
4. Servlet loaded → `jspInit()`.
5. For each request → `_jspService(req, res)`.
6. On destroy → `jspDestroy()`.

💡 Key Features:

✓ Implicit Objects (available in `_jspService()`):

- `request` , `response` , `session` , `application` , `out` , `pageContext` , etc.

✓ Scripting Elements:

- **Scriptlet:** `<% code %>`
- **Expression:** `<%= expr %>`
- **Declaration:** `<%! declarations %>`

✓ Expression Language (EL):

- `${param.name}` → `request.getParameter("name")`
- `${sessionScope.name}` → `session.getAttribute("name")`

✓ Directives:

- `<%@ page %>` , `<%@ include %>`
- Control page behavior, imports, error handling, threading, etc.

✓ JSP Actions:

- e.g., `<jsp:useBean>` , `<jsp:setProperty>` , `<jsp:getProperty>`

✓ JavaBeans in JSP:

- POJO with private props, public getters/setters, default constructor.
- Used to hold and transfer data and logic across pages.

✓ Tips to Improve Notes:

- Use headings and bullet points for better readability (especially around EL, directives, and lifecycle).
- Add diagrams to visualize lifecycle or EL resolution order.
- Emphasize real-world usage—e.g., JSTL or MVC usage with JSP.

✓ JSP Life Cycle (Detailed)

When a client sends a request to a `.jsp` page, the **Web Container (e.g., Tomcat)** is responsible for managing the life cycle of that JSP.

Here's the sequence:

🌀 1. Translation Phase

- **What happens?**
 - Web Container **translates the JSP file** (`.jsp`) into a **Servlet Java file** (`.java`).
- **Who does this?**
 - **Web Container** performs this action.

✓ **Depends on:** The `.jsp` file to generate the servlet source code.

2. Compilation Phase

- **What happens?**
 - The `.java` servlet is compiled into a `.class` file (bytecode).
- **Who does this?**
 - The **Java Compiler**, triggered by the Web Container.

✓ **Depends on:** The translated `.java` file from step 1.

3. Class Loading & Instantiation

- **What happens?**
 - Web Container loads the `.class` file and creates an object of the servlet.
- **Who does this?**
 - **Web Container**, using class loader.

✓ **Depends on:** The `.class` file.

4. Initialization Phase (`jspInit()` method)

- **What happens?**
 - Web Container calls the `jspInit()` method **only once** when the JSP is first loaded.
- **Who calls it?**
 - **Web Container** calls JSP's `jspInit()`.

✓ `jspInit()` depends on **Web Container** for invocation.

5. Request Processing (`_jspService(req, res)`)

- **What happens?**
 - For **every request**, the Web Container calls `_jspService()` method.
 - This is where your HTML + embedded Java runs.
- **Who calls it?**
 - **Web Container** calls the JSP's `_jspService()` method.

✓ `_jspService()` is called **once per request**.

! You CANNOT override `_jspService()` manually—it's auto-generated.

6. Destruction Phase (`jspDestroy()` method)

- **What happens?**
 - When the server is shutting down or the JSP is being undeployed, Web Container calls `jspDestroy()`.
- **Who calls it?**
 - **Web Container** calls JSP's `jspDestroy()`.

✓ `jspDestroy()` depends on Web Container to be called.

Summary Table: Who Depends on Whom?

Phase	Who Performs	Who Is Called	Who Depends on Whom
Translation	Web Container	—	Web Container depends on <code>.jsp</code> file
Compilation	Java Compiler (via WC)	—	Compiler depends on <code>.java</code> servlet file
Class Loading	Web Container	—	Depends on compiled <code>.class</code> file
Initialization	Web Container	<code>jspInit()</code>	JSP depends on WC to invoke <code>jspInit()</code>
Request Handling	Web Container	<code>_jspService(req, res)</code>	JSP depends on WC to invoke <code>_jspService()</code>
Destruction	Web Container	<code>jspDestroy()</code>	JSP depends on WC to invoke <code>jspDestroy()</code>

Key Relationships:

- **Web Container is the boss** – it controls the full life cycle.
- JSP is **passive** – it waits for the container to call its lifecycle methods.
- JSP's lifecycle methods (`jspInit` , `_jspService` , `jspDestroy`) are **called by the container**, not by the developer.

Who Is Calling Whom in JSP Life Cycle

Step	Caller	Method Called	Callee (Who Owns the Method)
1 Translation	Web Container	— (translates JSP to <code>.java</code>)	—
2 Compilation	Web Container → Java Compiler	— (compiles <code>.java</code> to <code>.class</code>)	—
3 Loading & Instantiation	Web Container	Constructor of JSP Servlet Class	JSP Servlet Class
4 Initialization	Web Container	<code>jspInit()</code>	JSP Servlet
5 Request Handling	Web Container	<code>_jspService(HttpServletRequest req, HttpServletResponse res)</code>	JSP Servlet
6 Destruction	Web Container	<code>jspDestroy()</code>	JSP Servlet

Important Notes:

✓ `jspInit()` :

- **Called by:** Web Container
- **Purpose:** One-time setup (e.g., DB connections, loading configs)

✓ `_jspService()` :

- **Called by:** Web Container
- **Purpose:** Handles every client request

- **Special note:** You **cannot override this manually** in JSP.

✓ `jspDestroy()` :

- **Called by:** Web Container
- **Purpose:** Cleanup before JSP is unloaded or container shuts down

Visualized Relationships (Plain Text)

```
Client Request
  ↓
Web Container
  |— Translates .jsp → .java
  |— Compiles .java → .class
  |— Loads .class → creates JSP instance
  |— Calls jspInit()      ← in JSP class
  |— Calls _jspService(req, res) ← in JSP class (on every request)
  |— Calls jspDestroy()   ← in JSP class (on shutdown)
```

So to summarize in one line:

- ✓ Web Container calls all key methods (`jspInit()`, `_jspService()`, `jspDestroy()`) inside the JSP class — the JSP itself doesn't call anything in the life cycle.

✓ Session Tracking using HttpSession + URL Rewriting

◆ Why use this technique?

To build a **cookie-independent web application** that can still maintain user sessions.

- Normally, **JSESSIONID** (session ID) is sent via a **cookie**.
- But if the client browser has cookies **disabled**, then we still need a way to track the session.
- **Solution:** Embed `JSESSIONID` directly in the **URL** using **URL rewriting**.

◆ What is URL Rewriting?

It's a technique to encode the session ID (JSESSIONID) into every outgoing URL (like in href, form action, sendRedirect) so the Web Container can still track the user's session without cookies.

Example encoded URL:

```
http://example.com/dashboard.jsp;jsessionid=98AB76DC09FF1234
```

◆ How does it work?

1. Web Container behavior:

- First, checks if `JSESSIONID` is coming via **cookie**.
- If **not found**, then looks for `JSESSIONID` **in the URL**.
- If found, it extracts the value and uses it to fetch the session (as usual).

2. Your responsibility as a developer:

- Use the following methods to **manually encode URLs** with session ID when cookies may not be available.

◆ Key APIs

Use Case	API Called	Called On	Returns
For links, form actions etc.	<code>String encodeURL(String url)</code>	<code>HttpServletResponse</code>	URL with session ID embedded
For <code>sendRedirect()</code>	<code>String encodeRedirectURL(String url)</code>	<code>HttpServletResponse</code>	Redirect URL with session ID

◆ Example:

```
// For anchor tag, form action, etc.  
String encodedURL = response.encodeURL("dashboard.jsp");  
  
// For redirection  
String redirectURL = response.encodeRedirectURL("welcome.jsp");  
response.sendRedirect(redirectURL);
```

◆ Who is calling whom?

Caller	Method Called	Purpose
Your Servlet/JSP Code	<code>response.encodeURL("url")</code>	Encode URL with session ID for links
Your Servlet/JSP Code	<code>response.encodeRedirectURL("url")</code>	Encode URL with session ID for redirect
Web Container	Looks for <code>JSESSIONID</code> in cookie or URL	To retrieve existing <code>HttpSession</code>

📌 NOTE:

- If the session ID **is already coming via a cookie**, the container **suppresses** URL rewriting (no need to duplicate session ID).
- That's why the container automatically **ignores encoding** if not necessary.

✅ 1. Request Parameters (`getParameter()`)

Feature	Description
Source	Sent by the client (from HTML form / URL query string).
Used in	<code>request.getParameter("key")</code>

Scope	Only current request.
Type	Always String .
Who sets	Client (browser).
Who reads	Servlet/JSP using <code>request.getParameter("key")</code>
Used for	Getting form data or query params.

 **Example:**

```
<form action="login.jsp" method="post">
  <input name="username">
</form>
```

```
<%= request.getParameter("username") %>
```

✓ 2. Attributes (`setAttribute()` / `getAttribute()`)

Scope	How to set	How to get	Lifetime
Request	<code>request.setAttribute("key", value)</code>	<code>request.getAttribute("key")</code>	One request
Session	<code>session.setAttribute("key", value)</code>	<code>session.getAttribute("key")</code>	Until session is active
Application	<code>application.setAttribute("key", value)</code>	<code>application.getAttribute("key")</code>	Until app/server restarts

Feature	Description
Set by	Servlet/JSP
Scope	Request / Session / Application
Type	Can be Object
Used for	Sharing data between JSPs/Servlets

Who is calling whom?

◆ `request.getParameter("key")`

- **Caller:** JSP/Servlet
- **Called method:** Internally handled by `HttpServletRequest` object.
- **Source:** Browser sends this data in form/URL.

◆ `request.setAttribute("key", value)`

- **Caller:** Servlet/JSP sets it
- **Then passed to:** Another JSP/Servlet using `RequestDispatcher.forward()`
- **Receiver:** Other component calls `request.getAttribute("key")` to read it

◆ `session.setAttribute("key", value)`

- **Caller:** Servlet/JSP
- **Stored in:** `HttpSession` object

- **Accessed by:** Any JSP/Servlet for the same client session

Summary Table

Feature	Parameter	Attribute
Who sets	Browser (Client)	JSP/Servlet (Server)
How to access	<code>getParameter()</code>	<code>getAttribute()</code>
Scope	Single Request	Request / Session / App
Data Type	Only String	Any Object
Purpose	Input from user	Share data internally

Day 9

What is JSTL?

JSTL is a **tag-based alternative to Java code in JSP**. It provides standard tags for:

- Redirection
- Setting/removing attributes
- Conditions
- Loops
- URL rewriting (session management)

 **Tomcat doesn't come with JSTL**, so you must **manually add the JSTL JAR** (e.g., `jstl-1.2.jar` in `WEB-INF/lib`).

Step-by-Step Usage

Step 1: Add JSTL jar

✓ Already done.

Step 2: Import the Core Tag Library

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

JSTL Tags Explained

1. Redirect (`<c:redirect>`)

✓ **Purpose:** Redirect to another resource like `response.sendRedirect()` but also supports session tracking if cookies are off.

```
<c:redirect url="next.jsp"/>
```

 **Behind the scenes:**

```
response.sendRedirect(response.encodeRedirectURL("next.jsp"));
```

 If cookies disabled:

```
next.jsp;jsessionid=ABC12345678
```

2. Set Attribute (`<c:set>`)

```
<c:set var="abc" value="${param.name}" />
```

✓ **Default scope:** `page`

 **Equivalent Java:**

```
pageContext.setAttribute("abc", request.getParameter("name"));
```

3. Remove Attribute (`<c:remove>`)

```
<c:remove var="abc" scope="session"/>
```

 **Equivalent Java:**

```
session.removeAttribute("abc");
```

4. If Condition (`<c:if>`)

```
<c:if test="${param.btn eq 'Withdraw'}">
  In Withdraw
</c:if>
```

 **Equivalent Java:**

```
if(request.getParameter("btn").equals("Withdraw")) {
  out.print("In Withdraw");
}
```

5. Switch-Case Style (`<c:choose>`)

```
<c:choose>
  <c:when test="${param.btn eq 'Withdraw'}">Withdraw</c:when>
  <c:when test="${param.btn eq 'Deposit'}">Deposit</c:when>
  <c:otherwise>Unknown action</c:otherwise>
</c:choose>
```

 **Equivalent Java:**

```
String btn = request.getParameter("btn");
if(btn.equals("Withdraw")) { ... }
else if(btn.equals("Deposit")) { ... }
else { ... }
```

6. Loop (`<c:forEach>`)

```
<c:forEach var="item" items="${sessionScope.items}">
    ${item}
</c:forEach>
```

Java Equivalent:

```
List<Item> items = (List<Item>) session.getAttribute("items");
for(Item item : items) {
    out.print(item);
}
```

You're referring to a common layered architecture pattern in Java web applications, particularly when using **JavaBeans**, **DAO (Data Access Object)**, and **JSP (Java Server Pages)**. Here's a clear breakdown of the dependency flow:

1. JavaBean depends on DAO:

- **JavaBean** is used to represent business logic or data models (like `Student`, `Employee`, `Book`, etc.).
- It interacts with the **DAO** layer to **fetch or persist data**.
- Example: A `StudentBean` might call `StudentDAO.getId(id)` to get data from the database.

2. JSP depends on JavaBean:

- **JSP** pages are used for **presentation** (UI).
- They use JavaBeans to **get or set values** and display them.
- You can use:

```
<jsp:useBean id="student" class="com.example.StudentBean" scope="request"/>
<jsp:getProperty name="student" property="name"/>
```

- JSP doesn't interact with DAO directly; instead, it depends on JavaBeans to get the data.

Full Flow:

```
[JSP] → uses → [JavaBean] → calls → [DAO] → interacts with DB
```

Would you like a code example for this structure?

Who is calling whom?

Tag	Internally uses	Called by
<code><c:redirect></code>	<code>response.sendRedirect()</code> + <code>encodeRedirectURL()</code>	JSTL Tag Handler
<code><c:set></code>	<code>pageContext.setAttribute()</code>	JSP engine
<code><c:remove></code>	<code>session.removeAttribute()</code> / <code>request.removeAttribute()</code>	JSP engine
<code><c:if></code>	Evaluates EL <code>\${}</code> as boolean	JSTL Core
<code><c:forEach></code>	<code>Iterator</code> or enhanced for-loop	JSTL Tag Handler

Should you learn JSTL before advanced topics like filters/session tracking?

Yes 

 **Reason:**

JSTL makes JSP cleaner and prepares you to understand:

- EL (Expression Language)
- MVC flow
- Writing less Java in JSP

Once you're comfortable with JSP + JSTL, you'll naturally understand how `HttpServlet`, `HttpSession`, and `filters` fit in.

Summary — Why Use JSTL?

- Makes JSP **cleaner** (no Java scriptlets)
- **Readable and maintainable**
- Fully supports **MVC structure**
- Works great with **EL** (Expression Language)
- Helps build scalable JSP apps

Hibernate

What is Hibernate?

- **Hibernate** is an **ORM (Object-Relational Mapping)** tool that helps Java applications automatically persist data in databases.
- It **implements JPA (Java Persistence API)**, which is a Java EE standard for persistence.

Why Use Hibernate?

1. **Open source & lightweight**
2. **Caching support:** L1 (Session) and L2 (SessionFactory level) improve performance.
3. **Auto table creation** with `hibernate.hbm2ddl.auto`.

4. **Simplifies joins** and queries.
 5. **DB Independence**: Use HQL/JPQL instead of DB-specific SQL.
 6. **No JDBC boilerplate code**: Connection handling, statement creation, result set processing is handled internally.
 7. **Connection pooling** is built-in (unlike JDBC).
 8. **Solves Object-Relational impedance mismatch**
 9. **Exception handling**: Converts checked SQLExceptions into unchecked HibernateExceptions.
 10. **Fluent API**: Supports method chaining.
-

✅ Hibernate Architecture (Key Components)

1. **SessionFactory** (Singleton)
 - Heavyweight, thread-safe.
 - One per DB/application.
 - Produces `Session` objects.
 - Can hold **L2 cache**.
 2. **Session**
 - Lightweight, not thread-safe.
 - Wraps around a DB connection.
 - Holds **L1 cache**.
 - Used to perform CRUD operations.
 3. **Configuration**
 - Reads `hibernate.cfg.xml`.
 - Used to configure Hibernate and build `SessionFactory`.
 4. **Transaction**
 - Used to manage commit/rollback.
-

✅ Hibernate Configuration File

- `hibernate.cfg.xml` : Bootstraps Hibernate.
- Contains DB connection details, dialect, and mapped classes.

✅ SessionFactory API

- **SessionFactory** is a heavyweight object used to create **Session** objects in Hibernate.
 - It is created **once per application** and is **thread-safe**.
 - Typically configured using `hibernate.cfg.xml`.
-

✅ openSession vs getCurrentSession

Feature	<code>openSession()</code>	<code>getCurrentSession()</code>
Returns	New <code>Session</code> each time	Same session bound to the current context
Closes Automatically	No, needs manual close	Yes, closed automatically at end of txn
Use case	Short-lived, manual session management	Recommended with declarative transactions
Example	<code>sessionFactory.openSession();</code>	<code>sessionFactory.getCurrentSession();</code>

To use `getCurrentSession()` , set:

```
<property name="hibernate.current_session_context_class">thread</property>
```

✓ POJO Class Example

Use annotations from `javax.persistence` :

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String firstName;
    private String lastName;

    @Column(unique = true, nullable = false)
    private String email;

    @Lob
    private byte[] image;

    @Enumerated(EnumType.STRING)
    private Role role;
}
```

✓ CRUD Operation Flow

```
Session session = factory.openSession(); // OR getCurrentSession()
Transaction tx = session.beginTransaction();

try {
    session.save(entity); // OR get/update/delete/load etc.
    tx.commit();
} catch (RuntimeException e) {
    tx.rollback();
    throw e;
```

```
} finally {  
    session.close(); // Returns connection to pool  
}
```

✓ Querying Using HQL/JPQL

- Object-oriented language (not SQL)

```
Query<User> query = session.createQuery("select u from User u", User.class);  
List<User> users = query.getResultList();
```

Would you like a diagram of the Hibernate architecture or a code demo for a specific CRUD operation?

Step-by-Step Guide

1. Create Maven Project

- Open your IDE (Eclipse or IntelliJ).
- Change the perspective to **Java**.
- Create a **Maven Project** with **Group ID**, **Artifact ID**, and **Packaging option** as `jar`.
- Click **Finish** to generate the default Maven structure.

2. Update `pom.xml`

- Replace the `<build>` and `<dependencies>` sections of the `pom.xml` with the following:

```
<build>  
  <plugins>  
    <plugin>  
      <groupId>org.apache.maven.plugins</groupId>  
      <artifactId>maven-compiler-plugin</artifactId>  
      <version>3.8.1</version>  
      <configuration>  
        <source>1.8</source>  
        <target>1.8</target>  
      </configuration>  
    </plugin>  
  </plugins>  
</build>  
  
<dependencies>  
  <dependency>  
    <groupId>org.hibernate</groupId>  
    <artifactId>hibernate-core</artifactId>  
    <version>5.4.27.Final</version>  
  </dependency>
```

```

<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>1.7.30</version>
</dependency>
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-simple</artifactId>
  <version>1.7.30</version>
</dependency>
<dependency>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.8.1</version>
</dependency>
</dependencies>

```

3. Update Project

- Right-click on the project → **Maven** → **Update Project**.
- Select **Force update** checkbox → Finish.

4. Import Existing Maven Project (Optional)

If you want to import an existing project:

- Copy the `test_hibernate_basic` project to your workspace folder.
- Go to **File** → **Import** → **Maven** → **Existing Maven Project**.
- Browse and select the `test_hibernate_basic` project → Finish.

5. Hibernate Configuration (`hibernate.cfg.xml`)

Edit your `hibernate.cfg.xml` file to include your database settings, like username, password, and JDBC URL. The configuration file should look something like this:

```

<!DOCTYPE hibernate-configuration PUBLIC "-//Hibernate/Hibernate Configuration DTD 3.0//EN" "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
  <session-factory>
    <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
    <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
  >
    <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/your_db_name</property>
    <property name="hibernate.connection.username">your_db_username</property>
    <property name="hibernate.connection.password">your_db_password</property>
    <property name="hibernate.hbm2ddl.auto">update</property>
    <property name="hibernate.show_sql">true</property>
    <property name="hibernate.format_sql">true</property>
  </session-factory>
</hibernate-configuration>

```

```

<property name="hibernate.current_session_context_class">thread</property>
<property name="hibernate.c3p0.min_size">5</property>
<property name="hibernate.c3p0.max_size">20</property>
<property name="hibernate.c3p0.timeout">300</property>
<property name="hibernate.c3p0.max_statements">50</property>
<property name="hibernate.c3p0.idle_test_period">3000</property>
<property name="hibernate.c3p0.acquire_increment">5</property>
</session-factory>
</hibernate-configuration>

```

6. Create **HibernateUtils** Class

This class creates a thread-safe singleton instance of **SessionFactory** :

```

package utils;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtils {
    private static final SessionFactory sessionFactory;

    static {
        try {
            sessionFactory = new Configuration().configure("hibernate.cfg.xml").addAnnotatedClasses(User.class).buildSessionFactory();
        } catch (Throwable ex) {
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSf() {
        return sessionFactory;
    }
}

```

7. Test Hibernate Bootstrapping (**TestHibernate.java**)

Create a class **TestHibernate** in the **tester** package to test Hibernate initialization:

```

import static utils.HibernateUtils.*;
import org.hibernate.*;

public class TestHibernate {
    public static void main(String[] args) {
        try(SessionFactory sf = getSf()) {
            System.out.println("Hibernate booted.....");
        } catch (Exception e) {

```

```

        e.printStackTrace();
    }
}
}

```

Run this as a **Java Application**. You should see the output `Hibernate booted.....` confirming Hibernate was initialized correctly.

8. POJO (Entity Layer)

Create a `User` POJO (Persistent Object) that will be mapped to a table. This will also enable Hibernate to automatically generate the table for you.

```

import javax.persistence.*;
import java.util.Date;

@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;

    @Column(name = "first_name")
    private String firstName;

    @Column(name = "last_name")
    private String lastName;

    @Column(name = "email")
    private String email;

    @Column(name = "password")
    private String password;

    @Column(name = "dob")
    private Date dob;

    @Enumerated(EnumType.STRING)
    @Column(name = "role")
    private Role role;

    @Lob
    @Column(name = "image")
    private byte[] image;
}

```

```
// Getters and setters
}
```

9. Update `hibernate.cfg.xml` to Include Mapping

Add the mapping for the `User` class:

```
<mapping class="User"/>
```

10. Create DAO Layer for User Registration

Create a `UserDAO` interface and its Hibernate-based implementation to insert records.

DAO Interface (`UserDAO.java`):

```
public interface UserDAO {
    String registerUser(User user);
}
```

Hibernate DAO Implementation (`UserDAOImpl.java`):

```
import org.hibernate.Session;
import org.hibernate.Transaction;

public class UserDAOImpl implements UserDAO {

    @Override
    public String registerUser(User user) {
        Transaction transaction = null;
        try (Session session = HibernateUtils.getSf().openSession()) {
            transaction = session.beginTransaction();
            session.save(user);
            transaction.commit();
            return "User registered successfully";
        } catch (Exception e) {
            if (transaction != null) transaction.rollback();
            e.printStackTrace();
            return "Error in user registration";
        }
    }
}
```

11. Create Main Tester for User Registration

Create a `Main` method to test user registration functionality:

```
import java.util.Date;

public class Main {
```

```

public static void main(String[] args) {
    User user = new User();
    user.setFirstName("John");
    user.setLastName("Doe");
    user.setEmail("john.doe@example.com");
    user.setPassword("password123");
    user.setDob(new Date());
    user.setRole(Role.USER);

    UserDao userDao = new UserDaoImpl();
    String result = userDao.registerUser(user);
    System.out.println(result);
}
}

```

12. Run the Application

Run the application. If everything is set up correctly, Hibernate will automatically create the `users` table in your database, and you will see a message indicating whether the user was registered successfully.

Why Use Maven?

- **Simplifies Builds:** Automates the process of compiling, testing, and packaging.
- **Dependency Management:** Downloads and manages project libraries automatically.
- **Standardization:** Provides a consistent structure and approach for Java projects.

Maven Project Structure:

- **src/main/java:** Source code.
- **src/main/resources:** Non-code resources like config files.
- **src/test/java:** Test code.
- **target:** Output directory for compiled code and artifacts.

Key Elements in pom.xml:

- **Group ID:** Unique identifier for the project group.
- **Artifact ID:** Name of the project.
- **Dependencies:** External libraries required for the project.
- **Build Plugins:** Tasks like compiling, testing, and packaging.

Common Maven Goals:

- `mvn clean`: Deletes the `target` directory.
- `mvn compile`: Compiles the source code.
- `mvn test`: Runs unit tests.

- `mvn package` : Packages the project into JAR/WAR.
- `mvn install` : Installs the package to the local repository.
- `mvn deploy` : Deploys the package to a remote repository.

How Maven Works:

1. Define your project in `pom.xml` (dependencies, plugins, etc.).
2. Run Maven commands (e.g., `mvn clean install`).
3. Maven downloads dependencies and executes phases like `compile` , `test` , and `package` .

Advantages:

- **Consistency**: Standardizes the build process.
 - **Automation**: Automates common tasks like compiling and testing.
 - **Efficient Dependency Management**: Handles external libraries automatically.
-

Day 10

✓ Entity States in Hibernate

1. Transient:

- Object is created using `new` but **not associated with Session**.
- Not stored in DB.
- Ex: `Student s = new Student();`

2. Persistent:

- Object is **associated with Session** and **mapped to a DB row**.
- Changes auto-synchronized.
- Ex: `session.save(s);`

3. Detached:

- Object was once persistent but **session is closed**.
- Not tracked anymore.
- Ex: `session.close();`

4. Removed:

- Object is **marked for deletion** from DB.
 - Ex: `session.delete(s);`
-

✓ CRUD Operations using Hibernate

- **Create**: `session.save(entity);`
- **Read**: `session.get(Entity.class, id);`

- **Update:**

```
entity.setName("Updated");
session.update(entity);
```

- **Delete:** `session.delete(entity);`

Typical structure:

```
Session session = sessionFactory.openSession();
Transaction tx = session.beginTransaction();
// perform operations
tx.commit();
session.close();
```

✅ JPQL (Java Persistence Query Language)

- Object-oriented version of SQL.
- Works with **entity names** not table names.

```
Query query = session.createQuery("from Student where name = :name");
query.setParameter("name", "Akshay");
List<Student> list = query.list();
```

Examples:

```
select s from Student s where s.name = 'Akshay'
update Student s set s.name = 'John' where s.id = 1
delete from Student s where s.id = 1
```

✅ Handling BLOBs in Hibernate

1. Entity Setup:

```
@Lob
private byte[] image;
```

1. Saving BLOB:

```
Student s = new Student();
s.setImage(Files.readAllBytes(Paths.get("photo.jpg")));
session.save(s);
```

1. Fetching BLOB:

```
Student s = session.get(Student.class, id);
Files.write(Paths.get("output.jpg"), s.getImage());
```

What Happens Upon Commit in Hibernate?

1. `session.flush()` is triggered

- This sends changes from Hibernate's L1 (first-level) cache to the database.

2. Automatic Dirty Checking

- Hibernate checks for any changes made to persistent objects.
- It compares the current state of the entities in the session with the database.

3. Based on the entity state:

- **New Persistent Entity:** Insert operation is executed.
- **Updated Entity:** Update operation is executed.
- **Deleted Entity:** If `session.delete()` was called, a Delete operation is executed.

4. `session.close()`

- First-level (L1) cache is destroyed.
- The database connection (pooled) is returned to the connection pool.

This process ensures that all changes made within a transaction are synchronized with the database when `commit()` is called.

What Happens Upon `rollback()` in Hibernate?

When a transaction is **rolled back**, Hibernate undoes all changes made during the transaction. Here's what happens step-by-step:

1. No `session.flush()`

- `rollback()` prevents `session.flush()` from being called automatically.
- That means changes made to entities in memory are **not pushed to the database**.

2. All DML operations are discarded

- Any **insert**, **update**, or **delete** operations that were pending are **discarded**.
- No SQL is executed on the database if it wasn't flushed manually before the rollback.

3. Persistent entities remain in memory

- The entities in the session may still reflect the changed state **in memory**.
- But the **database remains unchanged**.

4. L1 (First-Level) Cache is cleared upon `session.close()`

- If the session is closed after rollback, L1 cache is destroyed.
- Uncommitted changes in memory are lost.

5. DB Connection returned to pool

- The database connection is returned to the connection pool (if connection pooling is used).

In short:

```
rollback() =  
- Cancel transaction  
- No DB changes  
- Discard DML  
- Clear session  
- Return connection
```

Hibernate API Summary

0. SessionFactory API

- `openSession()` → Opens **new session**. Must be **explicitly closed**.
- `getCurrentSession()` → Reuses session bound to current thread (`current_session_context_class=thread`). **Auto-closed** on transaction boundary.

1. CRUD with Session API

- `save(Object)` → Persists **transient POJO**, returns **Serializable ID**. *(Deprecated in Hibernate 6)*
- `persist(Object)` → Persists transient object. Throws exception if ID is non-null.
- `saveOrUpdate(Object)` → Insert (null ID), Update (existing ID), throws exception (non-existing ID).
- `merge(Object)` → Handles **transient/detached**. Returns persistent copy. No exceptions like `saveOrUpdate` .

2. Data Retrieval

- `get(Class, Serializable)` → Returns **persistent POJO** or **null**.
- HQL: `"from BookPOJO"` or JPQL: `"select b from BookPOJO b"`
- Usage:

```
List<BookPOJO> books = session.createQuery("from BookPOJO", BookPOJO.class).getResultList();
```

3. Query with Parameters

```
String hql = "from BookPOJO b where b.price < :price and b.author = :author";  
List<BookPOJO> books = session.createQuery(hql, BookPOJO.class)  
    .setParameter("price", p)  
    .setParameter("author", a)  
    .getResultList();
```

4. Bulk Update

```
String hql = "update BookPOJO b set b.price = b.price - :disc where b.author = :au and b.publishDate < :dt";
int updated = session.createQuery(hql)
    .setParameter("disc", d)
    .setParameter("au", a)
    .setParameter("dt", date)
    .executeUpdate();
```

Note: Bypasses L1 cache, not suitable for cascading or optimistic locking.

5. Delete

- `session.remove(Object)` → Removes persistent entity from DB and L1 cache.
- HQL Delete:

```
int deleted = session.createQuery("delete Book b where b.publishDate < :d")
    .setParameter("d", new Date())
    .executeUpdate();
```

6. Update & Merge

- `update(Object)` → Requires **detached POJO** with **valid ID**.
 - Throws `TransientObjectException` (null ID), `StaleStateException` (ID doesn't exist), `NonUniqueObjectException` (conflict with session object).
- `merge(Object)` → Safe. Returns persistent copy, handles transient/detached.

7. Session Methods

- `evict(Object)` → Removes one object from L1 cache.
- `clear()` → Removes **all objects** from L1 cache.
- `flush()` → Syncs changes to DB before commit.
- `close()` → Clears L1 cache, returns DB connection to pool.
- `contains(Object)` → Checks if POJO is in session (L1 cache).
- `refresh(Object)` → Reloads data from DB.

8. Query Enhancements

- `setMaxResults(int)` / `setFirstResult(int)` → Used for **pagination**.
- `getSingleResult()` → One result. Throws if none/multiple.
- `scroll(ScrollMode)` → Enables **scrolling** through large results.
- **Pagination Example:**

```
Query<Book> q = session.createQuery("from Book b", Book.class)
    .setFirstResult(30)
    .setMaxResults(10);
List<Book> books = q.list();
```

9. Named Queries

```
@NamedQuery(name="Book.findById", query="select b from Book b where b.id = :id")
Book book = session.createNamedQuery("Book.findById", Book.class)
    .setParameter("id", 1)
    .getSingleResult();
```

10. Native SQL

```
List<BookPOJO> list = session.createSQLQuery("select * from books")
    .addEntity(BookPOJO.class)
    .list();
```

11. Criteria API

- Dynamically build complex queries:

```
CriteriaBuilder cb = session.getCriteriaBuilder();
CriteriaQuery<Book> cq = cb.createQuery(Book.class);
Root<Book> root = cq.from(Book.class);
cq.select(root).where(cb.equal(root.get("author"), "Author Name"));
```

12. Composite Keys

- Use `@Embeddable` key class (implements `Serializable`, overrides `equals()` and `hashCode()`).
- Annotate owning entity with `@EmbeddedId` .:

✓ Hibernate's Automatic Dirty Checking

Definition:

The process where Hibernate **automatically detects changes** made to **persistent objects** and **synchronizes** them with the database during a **flush** (usually at transaction **commit**) is called **Automatic Dirty Checking**.

How an Object Becomes Persistent

A POJO becomes **persistent** when:

1. You invoke:

- `session.save()`
- `session.persist()`
- `session.saveOrUpdate()`
- `session.merge()`

2. You load data using:

- `session.get()`
- `session.load()`

3. The object is returned as a result of a **JPQL** or **Criteria** query.

How Dirty Checking Works

- Hibernate tracks changes made to persistent objects in the **first-level cache (L1)**.
- When the **session is flushed**, Hibernate compares the current state of objects in memory with the state in the database.
- If differences are found, appropriate **DML (Data Manipulation Language)** queries (like `UPDATE`) are generated and executed.

Flushing the Session

- **Flush** = sync the state of session's objects with the database.
- Happens **automatically** at:
 - `transaction.commit()`
 - Executing certain queries (in some cases).
- Manual flush:

```
session.flush(); // not commonly needed
```

Key Points

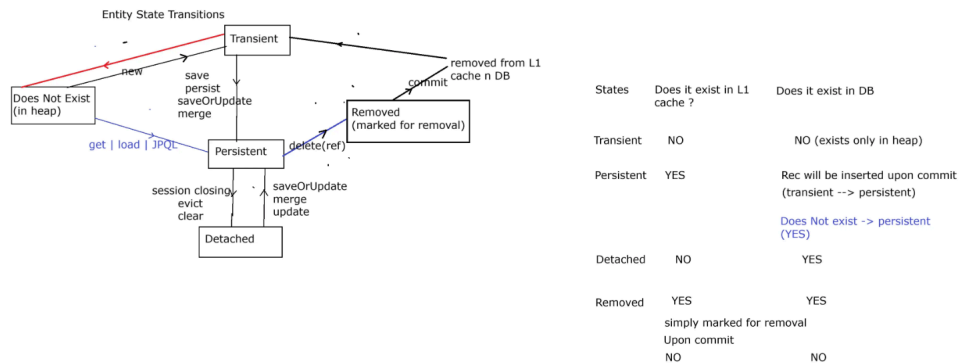
- No explicit `update()` is needed after modifying a persistent object.
- Hibernate does **not update the DB immediately** when the object is modified — changes are queued until **flush/commit**.
- This reduces redundant database operations and improves performance.

Hibernate Life Cycle

Hibernate manages the **life cycle of an entity (POJO)** across four major states:

1. **Transient**
2. **Persistent**
3. **Detached**
4. **Removed**

Let's explore each along with *who depends on whom* and *who calls whose method*.



1 Transient State

- **Definition:** Object is created using `new`, but **not associated with Hibernate Session** and **not stored in DB**.
- ☒ Who depends on whom:
 - The developer creates the POJO manually.
 - Hibernate doesn't know about it yet.
- ☒ Who calls whose method:
 - Developer calls: `new Entity()` (Hibernate is not involved).

```
Student s = new Student(); // transient
```

2 Persistent State

- **Definition:** Object is **associated with an open Hibernate Session**, and changes to it will be tracked and automatically synced to DB (via dirty checking).
- ☒ Who depends on whom:
 - Hibernate depends on **Session** to manage the object.
 - Entity object depends on Session to persist/track changes.
- ☒ Who calls whose method:
 - Developer calls:
 - `session.save(s)`

- `session.persist(s)`
- `session.saveOrUpdate(s)`
- `session.merge(s)`
- OR Hibernate loads the object via `session.get()` / `session.load()` / JPQL.
- Hibernate stores and manages the object in **L1 cache**.

```
Session session = factory.openSession();
session.beginTransaction();
session.save(s); // s becomes persistent
```

3 Detached State

- **Definition:** The session is closed or the object is manually evicted. The object is still in memory, but Hibernate **no longer tracks changes**.
-  Who depends on whom:
 - Developer must **manually reattach** the object if changes need to be saved.
 - Hibernate is not aware of object anymore.
-  Who calls whose method:
 - Hibernate calls `session.close()` or developer calls `session.evict(obj)`.
 - To reattach:
 - Developer calls `session.update(obj)` or `session.merge(obj)`.

```
session.close(); // s becomes detached
```

4 Removed State

- **Definition:** The object is scheduled for deletion from the database.
-  Who depends on whom:
 - Hibernate depends on transaction commit to complete deletion.
 - Entity must be in persistent state before removal.
-  Who calls whose method:
 - Developer calls: `session.delete(obj)`
 - Hibernate marks it for deletion; on `commit()`, DELETE query is fired.

```
session.delete(s); // s is marked for removal
```

Who Calls Whose Method — Summary

Action	Called By	Called On	Transition
--------	-----------	-----------	------------

<code>new Entity()</code>	Developer	POJO class	Transient state
<code>session.save(entity)</code>	Developer	Session	Transient → Persistent
<code>session.get()/load()</code>	Developer	Session	Persistent
<code>session.close()</code>	Developer	Session	Persistent → Detached
<code>session.evict(entity)</code>	Developer	Session	Persistent → Detached
<code>session.update(entity)</code>	Developer	Session	Detached → Persistent
<code>session.merge(entity)</code>	Developer	Session	Detached → Persistent (copy)
<code>session.delete(entity)</code>	Developer	Session	Persistent → Removed
<code>session.flush()</code> or <code>commit()</code>	Developer or Hibernate	Session	Flush to DB

Day 11

✓ What is Cascading in Hibernate?

Cascading refers to the ability of Hibernate to automatically **propagate operations** (like save, update, delete, etc.) from a **parent entity** to its **associated child entities**.

It helps manage complex entity relationships by reducing boilerplate and ensuring data consistency across entity associations.

🔧 How It Works

When you define relationships between entities (e.g., `@OneToMany`, `@ManyToOne`, etc.), you can specify a **cascade strategy** using the `cascade` attribute.

```
@OneToMany(mappedBy = "chosenCategory", cascade = CascadeType.ALL)
private List<BlogPost> posts = new ArrayList<>();
```

Now, performing an operation (e.g., `save`, `delete`) on the `Category` will automatically apply that operation to all its associated `BlogPost` objects.

🎯 CascadeType Enum Values (from `javax.persistence.CascadeType`)

CascadeType	Description	Use Case
<code>PERSIST</code>	Saves child entities when the parent is saved	Saving a new parent along with its children
<code>MERGE</code>	Updates children when parent is updated (merged)	Updating a detached graph of entities
<code>REMOVE</code>	Deletes child entities when the parent is deleted	Deleting a parent should delete all children
<code>REFRESH</code>	Refreshes children from DB when parent is refreshed	Ensures all data is current

DETACH	Detaches children from session when parent is detached	Detach entire graph from persistence context
ALL	Applies all of the above	For full cascading control

What is `orphanRemoval` in Hibernate?

`orphanRemoval = true` ensures that a **child entity is deleted from the database** if it is **removed from its parent's collection**.

Think of it as: "If the parent disowns a child, Hibernate should delete the child from the database automatically."

Where is it used?

It's used with these annotations:

- `@OneToMany`
- `@OneToOne`

```
@OneToMany(mappedBy = "category", cascade = CascadeType.ALL, orphanRemoval = true)
private List<Product> products = new ArrayList<>();
```

Default behavior (without `orphanRemoval`)

- When you remove a child (`Product`) from a parent's (`Category`) list, **only the association is broken**, but the **child remains in the database**.

Even with `CascadeType.REMOVE`, Hibernate doesn't assume a child should be deleted just because it's removed from the list — unless `orphanRemoval = true` is explicitly set.

When `orphanRemoval = true`

Now, if you do:

```
category.getProducts().remove(product); // now treated as an orphan
```

Hibernate will mark `product` for deletion when the transaction commits (via automatic dirty checking and flush).

Cascade vs `orphanRemoval`

Feature	Purpose	Deletes from DB if removed from parent list?
<code>CascadeType.REMOVE</code>	When parent is deleted, child is also deleted	✗ (only deletes when parent is deleted)
<code>orphanRemoval=true</code>	When child is removed from the collection	✓ (deletes orphan child automatically)

Advanced Hibernate: Inheritance, Associations, and Cascading

1. Inheritance Strategies in JPA/Hibernate

Hibernate provides **4 inheritance strategies** to manage the entity hierarchy effectively:

1. **MappedSuperclass** (`@MappedSuperclass`):

- Used for common fields in a parent class that are shared across multiple entities.
- No table is generated for this class. It acts as a base for other entities to inherit.
- Common fields could include `ID`, `createdDate`, `updatedAt`, and `version` for optimistic locking.

Use Case:

- Define common fields in the base class to promote reusability.
- Example:

```
@MappedSuperclass
public abstract class BaseEntity {
    @Id
    private Long id;
    @CreationTimestamp
    private LocalDateTime createdDate;
    @UpdateTimestamp
    private LocalDateTime updatedAt;
    @Version
    private Integer version;
}
```

2. **Associations between Entities (HAS-A Relationship):**

- A weaker form of association that allows one entity to "have" another entity as part of its lifecycle.
- Typically applied in **one-to-many** and **many-to-one** relationships.

Example:

- **Restaurant 1 $\longleftrightarrow$ * FoodItem**
 - `Restaurant` is the parent, containing multiple food items.
 - `FoodItem` is the child, owning the foreign key (`Restaurant_id`).

Code:

```
@Entity
public class Restaurant extends BaseEntity {
    private String name;
    private String address;
    private String city;
    private String description;
    @OneToMany(mappedBy = "chosenRestaurant", cascade = CascadeType.ALL)
    private List<FoodItem> foodItems = new ArrayList<>();
}
```

```

}

@Entity
public class FoodItem extends BaseEntity {
    private String itemName;
    private String itemDescription;
    private boolean isVeg;
    private int price;
    @ManyToOne
    @JoinColumn(name = "Restaurant_id", nullable = false)
    private Restaurant chosenRestaurant;
}

```

Points to Remember:

- **Owning Side:** The entity that contains the foreign key (e.g., `FoodItem`).
- **Inverse Side:** The entity that does **not** contain the foreign key (e.g., `Restaurant`).

Uni-directional vs. Bi-directional Associations:

- **Uni-directional:** The relationship is navigable only from one side (e.g., `Course 1 --> * Student`).
- **Bi-directional:** Both sides of the relationship are navigable (e.g., `Restaurant 1 <--> * FoodItem`).

Bi-directional Setup:

- **Restaurant (parent):** `@OneToMany(mappedBy = "chosenRestaurant")`
- **FoodItem (child):** `@ManyToOne`

2. Cascading in Hibernate

Cascading ensures that operations performed on one entity automatically propagate to its associated entities. It simplifies managing the entity lifecycle.

Cascading Operations:

- **PERSIST:** Save child entities when saving the parent.
- **MERGE:** Update child entities when updating the parent.
- **REMOVE:** Delete child entities when deleting the parent.
- **REFRESH:** Refresh child entities when refreshing the parent.
- **DETACH:** Detach child entities when detaching the parent.
- **ALL:** Applies all the above cascading operations.

Example with `@OneToMany`:

```

@OneToMany(mappedBy = "chosenRestaurant", cascade = CascadeType.ALL)
private List<FoodItem> foodItems = new ArrayList<>();

```

With `cascade = CascadeType.ALL`, saving, updating, or deleting a `Restaurant` will automatically affect the associated `FoodItems`.

3. Orphan Removal

`orphanRemoval` ensures that **orphaned** child entities (i.e., those removed from a relationship) are deleted from the database.

Use Case:

- You want a child entity to be automatically deleted when it is removed from the parent entity's collection.

```
@OneToMany(mappedBy = "chosenRestaurant", cascade = CascadeType.ALL, orphanRemoval = true)
private List<FoodItem> foodItems = new ArrayList<>();
```

If you remove a `FoodItem` from the `foodItems` collection, it will be deleted from the database.

4. LazyInitializationException and Fetching Strategies

Hibernate uses **lazy loading** for associations like `one-to-many` and `many-to-many`, meaning associated entities are not loaded until explicitly accessed. This leads to the `LazyInitializationException` when you try to access the association outside of an active session.

Solution:

- Use `EAGER` fetching for `@OneToMany` if you want to load the association immediately.

```
@OneToMany(mappedBy = "chosenRestaurant", fetch = FetchType.EAGER)
private List<FoodItem> foodItems = new ArrayList<>();
```

- Alternatively, you can use **JPQL** with `JOIN FETCH` to fetch the related entities in a single query.

```
String jpql = "SELECT r FROM Restaurant r JOIN FETCH r.foodItems WHERE r.name = :name";
List<Restaurant> restaurants = entityManager.createQuery(jpql, Restaurant.class)
    .setParameter("name", "Pizza Hut")
    .getResultList();
```

This will avoid lazy loading issues by fetching the associated `foodItems` with the `Restaurant` in one query.

5. Delete Operations with Cascade

When deleting a parent entity, you need to delete child entities first if there's a cascade delete involved. You can define cascade operations to propagate the delete action.

Example: Deleting a `Restaurant` with its associated `FoodItem` :

```
Restaurant restaurant = entityManager.find(Restaurant.class, restaurantId);
entityManager.remove(restaurant); // will also remove associated FoodItems if cascade is set.
```

6. Helper Methods for Managing Associations

To streamline adding/removing children from the collection, consider adding helper methods in the entity class to manage associations.

Example:

```

public void addFoodItem(FoodItem foodItem) {
    foodItems.add(foodItem);
    foodItem.setChosenRestaurant(this);
}

public void removeFoodItem(FoodItem foodItem) {
    foodItems.remove(foodItem);
    foodItem.setChosenRestaurant(null);
}

```

These methods encapsulate the logic for adding/removing child entities and ensure consistency in the entity relationships.

Conclusion

By implementing inheritance, associations, cascading, and orphan removal in Hibernate, you can manage complex entity relationships with minimal effort. Understanding the different fetching strategies and how to handle lazy loading, along with defining helper methods for managing collections, will make your entity management more robust and efficient.

Day 12

Hibernate Performance Tips:

1. One-to-Many Bi-directional Association:

- Always configure the **one side** as the inverse side and the **many side** as the owning side (FK containment).

2. Uni-directional One-to-Many Association:

- Avoid configuring uni-directional one-to-many associations to prevent multiple select queries.

3. Many-to-One Uni-directional Association:

- Prefer uni-directional many-to-one associations for better performance.

4. One-to-One Associations:

- Configure either uni or bi-directional based on your project's data access needs.

5. Many-to-Many Associations:

- Configure either uni or bi-directional based on your project's data access needs.

Topics for Practice:

1. Entity Associations:

- Practice configuring associations such as `@MappedSuperclass`, `@OneToMany` (mappedBy, cascade, orphanRemoval), `@ManyToOne`, and `@JoinColumn`.

2. Validation:

- Ensure a blogger can't comment on their own blog post.

- Allow multiple comments from the same user on a post.

3. Comment Operations:

- Post a new comment, including input like comment text, rating, commenter ID, and post ID.
- Display all comments for a specific post using JPQL.

4. Category Details:

- Retrieve and display category details and associated posts.
- Handle `LazyInitializationException` when accessing related data.

5. One-to-One Association:

- Configure a one-to-one relationship between `User` and `Address`, and manage cascading and FK column names.

6. Many-to-Many Association:

- Implement uni-directional many-to-many between `Tags` and `Posts` using `@ManyToMany` and `@JoinTable`.

Entity vs Value Types:

- **Entity Types:** Have their own lifecycle (e.g., `User`, `BlogPost`).
- **Value Types:** Do not have an independent lifecycle (e.g., `AdharCard`, `PaymentCard`).

Example: A `User` has a value type like `AdharCard` (embeddable).

JPA Annotations:

1. **@Transient:** Marks a field to be excluded from persistence (not stored in the database).
2. **@Temporal:** Specifies date/time precision for `java.util.Date`, `Calendar`, etc.

In a **One-to-Many** or **Many-to-One** relationship, the **owning side** is the side that contains the foreign key (FK) column and is responsible for maintaining the relationship in the database.

1. One-to-Many Relationship:

- **Owning side:** The **Many** side (child) is the owning side because it contains the foreign key (FK) reference to the parent entity.
- **Inverse side:** The **One** side (parent) is the inverse side, which doesn't hold the foreign key.

Example:

```
@Entity
public class Category {
    @OneToMany(mappedBy = "category") // inverse side
    private List<Product> products;
}
```

```
@Entity
public class Product {
    @ManyToOne // owning side
```

```
@JoinColumn(name = "category_id") // FK column
private Category category;
}
```

- In this case, `Product` (many side) is the owning side because it has the foreign key `category_id`.

2. Many-to-One Relationship:

- **Owning side:** The **Many** side (child) is the owning side because it contains the foreign key (FK) reference to the parent entity.
- **Inverse side:** The **One** side (parent) is the inverse side.

Example:

```
@Entity
public class Order {
    @ManyToOne // owning side
    @JoinColumn(name = "customer_id") // FK column
    private Customer customer;
}

@Entity
public class Customer {
    @OneToMany(mappedBy = "customer") // inverse side
    private List<Order> orders;
}
```

- Here, `Order` (many side) is the owning side because it has the foreign key `customer_id`.

General Rule:

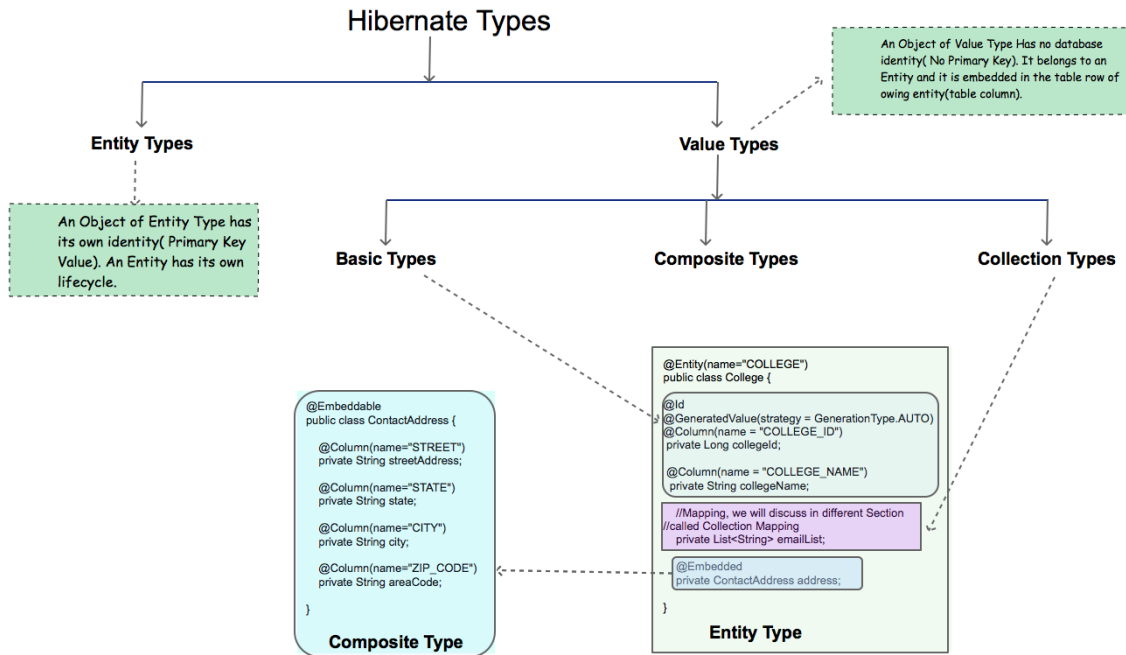
- In a **One-to-Many** relationship, the **Many** side owns the relationship (contains the foreign key).
- In a **Many-to-One** relationship, the **Many** side still owns the relationship because it holds the foreign key.

How to Identify the Owning Side:

1. Look for the `@JoinColumn` annotation.
2. The side with the `@JoinColumn` (which contains the foreign key) is the owning side.
3. The side with the `mappedBy` attribute is the inverse side.

Why It Matters:

- The owning side is responsible for updating the relationship in the database (like adding/removing associations).
- The inverse side just reflects the association but doesn't directly modify the database relationship.



Entity Types vs Value Types

Entity Types:

1. **Definition:** Entities have their own database identity, usually through a primary key.
2. **Lifecycle:** Entities have an independent lifecycle and can exist without any other entity.
3. **Database Representation:** Each entity instance is stored in its own row with a primary key.
4. **Mandatory Annotations:**
 - `@Entity` : Marks the class as an entity.
 - `@Id` : Specifies the primary key for the entity.

Example:

```

@Entity
public class College {
    @Id
    private Long id;
    private String name;
}
    
```

Value Types:

1. **Definition:** Value types do not have their own identity (i.e., no primary key). They are defined within an entity and do not exist independently.
2. **Ownership:** They belong to an entity and are embedded within it.
3. **Lifecycle:** The lifespan of a value type is tied to the lifecycle of the owning entity.
4. **Annotations:**

- `@Embeddable` : Marks a class as a value type that can be embedded in an entity.
- `@Embedded` : Used in an entity to indicate that a value type is embedded.
- `@AttributeOverride` : Used to override column mappings for embedded fields.

Example:

```
@Embeddable
public class Address {
    private String street;
    private String city;
}

@Entity
public class User {
    @Id
    private Long id;
    @Embedded
    private Address address;
}
```

Types of Value Types:

1. Basic Value Types:

- These are simple types like `String` , `Integer` , `Boolean` , etc.
- They map to single database columns.
- Hibernate supports built-in basic types.

Example:

```
@Entity
public class User {
    @Id
    private Long id;
    private String name;
    private Integer age;
}
```

2. Composite Value Types (Embedded Types):

- A composite value type (also called embedded type) is a class that does not own its identity but is used as part of another entity.
- It is marked with `@Embeddable` and embedded in an entity using `@Embedded` .

Example:

```
@Embeddable
public class Address {
```

```

    private String street;
    private String city;
}

@Entity
public class Student {
    @Id
    private Long id;
    @Embedded
    private Address address;
}

```

3. Collection Value Types:

- Hibernate allows collections of value types (both basic and composite types) to be persisted.
- The collection must be stored in a separate table, with a join to the owning entity.
- Collections of embeddable types and basic types are common use cases.

Example for Collection of Embeddable Types:

```

@Entity
public class User {
    @Id
    private Long id;
    @ElementCollection
    @CollectionTable(name = "CONTACT_ADDRESS", joinColumns = @JoinColumn(name = "USER_ID"))
    @AttributeOverride(name = "street", column = @Column(name = "STREET_ADDRESS"))
    private List<Address> addresses;
}

```

Example for Collection of Basic Types:

```

@Entity
public class User {
    @Id
    private Long id;
    @ElementCollection
    @CollectionTable(name = "CONTACTS", joinColumns = @JoinColumn(name = "USER_ID"))
    @Column(name = "CONTACT_NO")
    private Collection<String> contacts;
}

```

Additional Notes on Annotations:

- **@AttributeOverride** : Used to override column names when embedding a value type in an entity.

- Useful when you need different column names in different tables for the same embedded value type field.

Example:

```
@Embedded
@AttributeOverride(name = "street", column = @Column(name = "STREET_ADDRESS"))
private Address address;
```

- **@Embeddable** : Used to mark a class as a value type that can be embedded into other entities. The class does not have its own identity and cannot exist independently.

Example:

```
@Embeddable
public class Address {
    private String street;
    private String city;
}
```

Summary:

- **Entities** have their own primary key and lifecycle, marked by **@Entity** and **@Id** .
- **Value Types** do not have their own primary key and are embedded in entities, marked by **@Embeddable** and **@Embedded** .
- You can persist collections of basic or composite types using **@ElementCollection** .

LazyInitializationException in Hibernate

Why does Hibernate throw **LazyInitializationException** ?

`org.hibernate.LazyInitializationException` occurs when an uninitialized proxy object (a placeholder for a collection or an entity) is accessed outside the session scope, or when Hibernate tries to load an associated entity or collection that was not fetched.

Hibernate uses **lazy loading** by default for **one-to-many** , **many-to-many** associations, meaning the associated entities are not fetched immediately when the parent entity is loaded. Instead, they are fetched only when accessed. However, if you try to access these associations after the session is closed (i.e., the entity is detached), Hibernate cannot initialize them and throws the exception.

Default Fetching Policies in JPA/Hibernate:

- **One-to-One**: EAGER (default)
- **One-to-Many**: LAZY (default)
- **Many-to-One**: EAGER (default)
- **Many-to-Many**: LAZY (default)

Explanation of **LazyInitializationException** with an Example

Consider the following **Category** entity with a **one-to-many** relationship to **BlogPost** :

```

@Entity
public class Category {
    @Id
    private Long id;

    private String name;

    @OneToMany(mappedBy = "chosenCategory", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<BlogPost> posts = new ArrayList<>();
}

```

Here:

- The `Category` has a `one-to-many` relationship with `BlogPost`.
- The `@OneToMany` relationship uses **lazy fetching** by default.

Problem:

If you load a `Category` entity and then access its `posts` collection after the session is closed, Hibernate will throw a `LazyInitializationException` because the `posts` collection is represented by a proxy that requires a database query to be loaded, but there is no open session to perform this query.

Solutions to Avoid LazyInitializationException

1. Change Fetch Type to EAGER (Not recommended for `one-to-many`)

You can change the fetch type to `EAGER` to force Hibernate to load the associated `BlogPost` entities immediately when the `Category` is loaded.

```

@OneToMany(mappedBy = "chosenCategory", cascade = CascadeType.ALL, orphanRemoval = true, fetch = FetchType.EAGER)
private List<BlogPost> posts = new ArrayList<>();

```

Disadvantages:

- **Not recommended:** Eager loading can result in performance issues, especially with `one-to-many` or `many-to-many` relationships. It forces Hibernate to load all associated entities even when they are not needed, which can lead to performance bottlenecks.
- **Select N+1 Problem:** If there are many `BlogPost` entries for each `Category`, Hibernate will execute one query to load the categories and then separate queries to load each associated `BlogPost`, which is inefficient.

When to use: This solution is only suitable when the number of associated entities is small and you need them immediately.

2. Access the Collection Within the Session Scope

To avoid accessing the collection after the session is closed, ensure you access the collection while the session is still open.

Solution:

Access the `posts` collection while the entity is still in the session scope (i.e., before the session is closed).

```
@Transactional
public Category getCategoryWithPosts(Long categoryId) {
    Category category = categoryRepository.findById(categoryId); // Inside session
    category.getPosts().size(); // Accessing the collection within the session
    return category;
}
```

Disadvantages:

- Hibernate fires **multiple queries**: One query for loading the category and then another query for each associated `BlogPost`.
- **Less efficient** than a single query that loads everything in one go, especially with large collections.

3. Use **JOIN FETCH** in JPQL to Avoid Lazy Initialization Issues

The **best solution** to avoid `LazyInitializationException` and the **N+1 select problem** is to use **JOIN FETCH** in your JPQL query. This forces Hibernate to fetch the associated collection in a single SQL query using a **join**.

Example (for one-to-many relationship):

```
String jpql = "select c from Category c join fetch c.posts where c.name = :categoryName";
Category category = entityManager.createQuery(jpql, Category.class)
    .setParameter("categoryName", "Technology")
    .getSingleResult();
```

Advantages:

- **Single query**: Hibernate will use a **single SQL JOIN** to fetch the `Category` and its `BlogPost` entities, avoiding the N+1 problem.
- **No LazyInitializationException**: Since the collection is fetched immediately within the same session, you can safely access it outside the session without the exception.

For left outer join:

```
String jpql = "select c from Category c left join fetch c.posts where c.name = :categoryName";
Category category = entityManager.createQuery(jpql, Category.class)
    .setParameter("categoryName", "Technology")
    .getSingleResult();
```

This solution is efficient because it avoids multiple queries and still provides all the necessary data in a single query.

Summary of Solutions:

1. **Change Fetch Type to EAGER** (Not recommended for `one-to-many`): Forces immediate loading of associated entities, but may cause performance problems with large collections.
2. **Access the Collection Within the Session Scope**: Ensures you access lazy-loaded collections while the session is open, but may result in multiple queries being fired.
3. **Use `JOIN FETCH` in JPQL**: Most recommended solution to fetch associations in a single query without `LazyInitializationException` or N+1 select issues.

Day 13

Spring Framework Overview

Why Spring?

Spring simplifies Java development and promotes best practices like **loose coupling** and **easy testability**. Here's why you would use it:

1. **Simplifies Development**: Spring provides a comprehensive framework for building enterprise-level applications, simplifying configuration and setup.
2. **Loose Coupling**: Using **Dependency Injection** (DI) and **Inversion of Control** (IoC), Spring enables the decoupling of components and their dependencies.
3. **Design Patterns Support**: Spring offers implementations of common design patterns like **Singleton**, **Factory**, **Proxy**, and **MVC**.
4. **Reduces Boilerplate Code**: Spring eliminates repetitive code (like managing connections, handling transactions, etc.), improving maintainability.
5. **Lightweight**: Spring applications are light and easily configurable, especially when compared to heavyweight frameworks.
6. **Excellent Database Integration**: Supports JDBC and ORM (Hibernate) for seamless database interaction with transaction management.
7. **Modular Design**: Spring's modular architecture allows you to pick and choose only the necessary modules.
8. **Integration with Other Frameworks**: Easily integrates with other technologies like **Hibernate**, **Struts**, and **EJB**.
9. **Aspect-Oriented Programming (AOP)**: Supports separation of cross-cutting concerns, such as logging or security, from business logic.
10. **Testing Support**: Built-in support for unit testing and integration testing using frameworks like **JUnit** and **Mockito**.
11. **Microservices Support**: Spring Boot simplifies the creation of **microservices** based applications.
12. **Secure Web Development**: Provides built-in tools to secure web apps and RESTful web services.

What is Spring?

1. **Spring is a Container**: It manages the lifecycle of beans, which are just plain Java objects (POJOs) that Spring can manage.

- A **Spring Bean** is a Java object whose lifecycle is managed by the Spring container. Common examples of beans are controllers, services, and DAOs in web applications.
2. **Framework of Frameworks:** Spring provides various modules that can be used together or separately. These modules include frameworks like **Spring MVC**, **Spring Boot**, **Spring Data**, **Spring Security**, etc.
 3. **Modular and Extensible:** Spring is a highly modular framework, allowing developers to include only the necessary components.
 4. **Spring's Role in Dependency Injection (DI):**
 - DI allows for **loose coupling**. Objects don't create their dependencies, but rather receive them from the container. This makes code easier to test and maintain.
 5. **Spring Framework's Design Patterns:** It offers implementations of various design patterns like **Singleton**, **Factory**, **Proxy**, **Front Controller**, **MVC**, and **Template Method**.
-

Dependency Injection (DI) / Inversion of Control (IoC)

1. Tight Coupling Issue:

- Traditional Java code often tightly couples dependent objects and their dependencies (e.g., DAOs, utilities). Changing one dependency requires changes in all dependent objects, making maintenance harder.

2. What is DI?

- **Dependency Injection** is a design pattern where an object's dependencies are provided to it rather than it creating the dependencies itself.
- Spring uses **Inversion of Control (IoC)** to manage the creation and injection of dependencies.
- **IoC:** The container, not the dependent object, controls the lifecycle and injection of dependencies.

Example:

```
@Autowired
private IUserService userService;
```

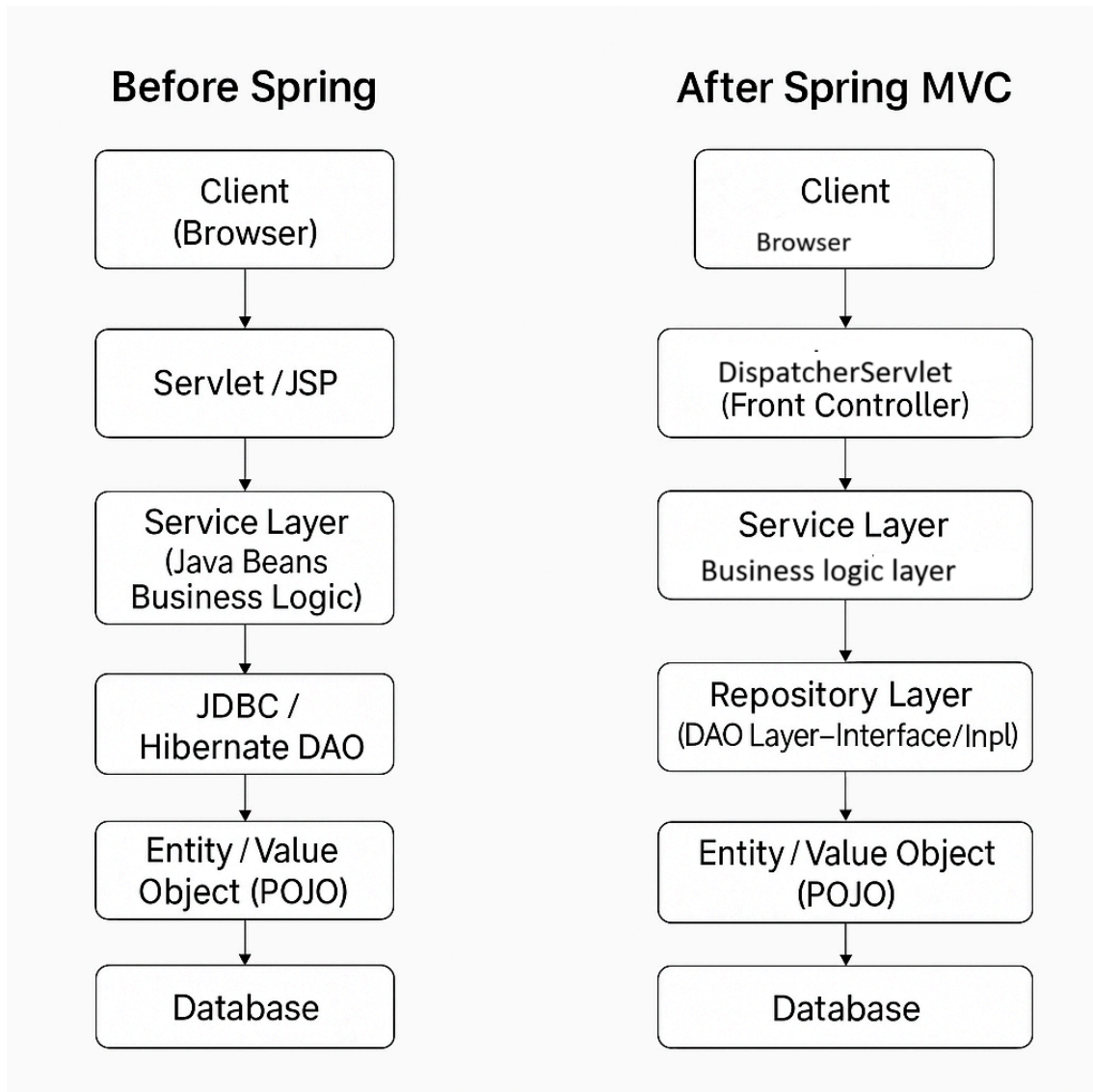
- The `@Autowired` annotation allows Spring to inject the `IUserService` implementation into the `UserController` without manually instantiating it.

Spring Bean Lifecycle

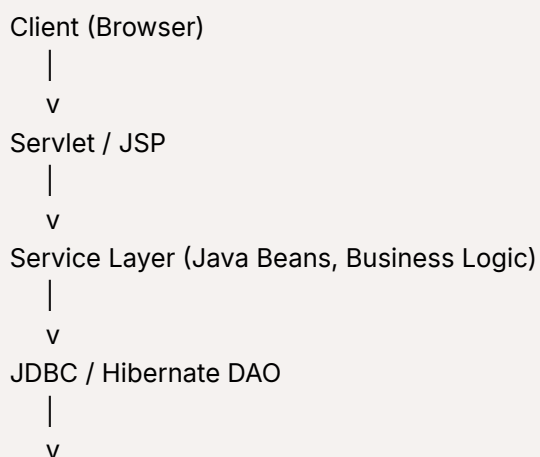
The lifecycle of a Spring Bean is defined by several stages:

1. **Instantiation:** Spring creates a bean using the provided class name.
2. **Dependency Injection:** Spring injects dependencies into the bean (via constructor or setter injection).
3. **Post-Processing:** After the bean is initialized but before it is used, **bean post-processors** may modify the bean.
4. **Initialization:** If the bean defines an `init-method`, it is called at this point.

5. **Destruction:** When the application context is closed, beans are destroyed. If the bean defines a `destroy-method`, it is called at this point.



1. Before Spring (Traditional JSP/Servlet Architecture)



Entity / Value Object (POJO)
|
v
Database

2. After Spring MVC

Client (Browser)
|
v
DispatcherServlet (Front Controller)
|
v
Controller / RestController
|
v
Service Layer (Business Logic, Transaction)
|
v
Repository Layer (DAO Layer - Interface/Impl)
|
v
Entity / Value Object (POJO)
|
v
Database

Key Differences:

Feature	Before Spring	After Spring MVC
Request Handling	Servlet/JSP	DispatcherServlet (Front Controller)
Business Logic Layer	Java Beans manually linked	@Service with DI (Spring-managed)
DAO Layer	JDBC / Hibernate	@Repository using Spring Data JPA
Entity Management	Manual POJOs	JPA Entities with annotations
Loose Coupling	No	Yes (via Dependency Injection)
Testability	Low	High (due to layering and DI)

Bean Wiring

Spring allows you to wire dependencies in two ways:

1. Explicit Wiring (XML-based):

- **Setter-based DI:**

```
<bean id="userController" class="com.example.UserController">
  <property name="userService" ref="userServiceBean"/>
</bean>
```

- **Constructor-based DI:**

```
<bean id="userController" class="com.example.UserController">
  <constructor-arg ref="userServiceBean"/>
</bean>
```

2. Implicit Wiring (Annotation-based):

- **@Autowired** is used to inject dependencies automatically.

```
@Autowired
private IUserService userService;
```

Spring API and BeanFactory

- **BeanFactory:** The core container in Spring, used for managing beans.

```
BeanFactory factory = new XmlBeanFactory(new ClassPathResource("beans.xml"));
MyBean myBean = (MyBean) factory.getBean("myBean");
```

- **ApplicationContext:** Extends `BeanFactory` and provides more advanced features like event handling and internationalization.

Types of Bean Scopes in Spring

1. **Singleton:** (Default) One instance of the bean is shared across the entire application.
2. **Prototype:** A new instance of the bean is created each time it's requested.
3. **Request:** A new bean instance is created for each HTTP request.
4. **Session:** A new bean instance is created for each HTTP session.
5. **Global Session:** Used in portlet-based applications.

Bean Lifecycle Methods

1. **init-method**: Custom initialization logic.

```
<bean id="myBean" class="com.example.MyBean" init-method="initialize"/>
```

2. **destroy-method**: Custom destruction logic.

```
<bean id="myBean" class="com.example.MyBean" destroy-method="cleanup"/>
```

Summary of Key Spring Concepts

- **Spring Container** manages beans and their lifecycle.
- **DI** (Dependency Injection) decouples components, improving testability and maintainability.
- Spring provides multiple ways to configure beans: **XML-based**, **Annotation-based**, and **Java-based configuration**.
- **AOP** (Aspect-Oriented Programming) allows the separation of concerns like logging, security, and transaction management.
- Spring enables building loosely-coupled, modular applications with **various scopes** (singleton, prototype, etc.), simplifying the development of web applications and microservices.

By following the Spring approach, you ensure that your application is easy to maintain, extend, and test.

Who is Dependent on Whom for What?

Dependent Object	Depends On (Dependency)	For What Purpose?
Controller Layer (e.g., <code>UserController</code>)	Service Layer (e.g., <code>UserService</code>)	To handle business logic requested by web layer or REST clients.
Service Layer (e.g., <code>ServiceImpl</code>)	DAO Layer (e.g., <code>UserDao</code>)	To interact with the database or persistence logic.
DAO Layer (e.g., <code>DaoImpl</code>)	ORM/JDBC Tools (e.g., <code>SessionFactory</code> , <code>JdbcTemplate</code>)	To establish DB connection and execute queries.
Any Class with <code>@Autowired</code> Fields	Spring Container (SC)	To receive the required dependencies at runtime without instantiating them manually.

Example in Real Code

1. Controller depends on Service

```
@RestController
public class UserController {

    @Autowired
    private IUserService userService; // dependent on IUserService for business logic
}
```

2. Service depends on DAO

```
@Service
public class UserServiceImpl implements IUserService {

    @Autowired
    private IUserDao userDao; // dependent on DAO for DB operations
}
```

3. DAO depends on ORM Utility

```
@Repository
public class UserDaoImpl implements IUserDao {

    @Autowired
    private SessionFactory sessionFactory; // dependency for getting DB session
}
```

✓ Final Summary

- **Controllers** depend on **Services** to perform business logic.
- **Services** depend on **DAOs** to interact with the database.
- **DAOs** depend on **JDBC/Hibernate utilities** (like `SessionFactory`) to execute queries.
- All of these objects rely on **Spring Container** to inject their dependencies (via **Dependency Injection**).

Would you like a simple UML-style diagram showing these relationships?

Let's analyze **who is calling whose method for what purpose** in a typical **Spring application (MVC or layered architecture)**.

🔄 Flow of Control in Spring (Who Calls Whom, and Why)

Caller (Who)	Callee (Whom)	Method Called	Purpose
Client (browser/postman)	Controller (<code>@RestController</code>)	<code>handleRequest()</code>	Sends request (e.g., GET/POST) to controller to perform some operation
Controller	Service (<code>@Service</code>)	<code>performBusinessLogic()</code>	Delegates business logic (e.g., registration, login, etc.)
Service	DAO (<code>@Repository</code>)	<code>save()</code> , <code>findById()</code> , <code>delete()</code>	Delegates database access or manipulation
DAO	Hibernate/JDBC (<code>SessionFactory</code> , <code>JdbcTemplate</code>)	<code>getCurrentSession()</code> , <code>executeUpdate()</code> , <code>query()</code>	Executes SQL/JPQL to interact with database
Spring Container	All Beans (Controller, Service, DAO)	<code>constructor/setter</code>	Injects dependencies automatically (via DI) at runtime

🔄 Example Trace of a Web Request

Suppose a client sends a POST request to `/register`.

1. **Client** sends HTTP POST → `/register`
2. `UserController.registerUser()` is called
 - Calls `userService.registerUser(user)`
3. `UserServiceImpl.registerUser()` is called

➤ Calls `userDao.saveUser(user)`

4. `UserDaoImpl.saveUser()` is called

➤ Calls `sessionFactory.getCurrentSession().save(user)`

5. Hibernate performs insert in **DB**

Detailed Method Call Mapping

Layer	Method	Who Calls This?	Why It's Called
Controller (<code>UserController</code>)	<code>registerUser()</code>	HTTP Client	To trigger a registration
Service (<code>UserServiceImpl</code>)	<code>registerUser()</code>	Controller	To perform business validations, logic
DAO (<code> UserDaoImpl</code>)	<code>saveUser()</code>	Service	To persist user in DB
Hibernate	<code>save()</code>	DAO	To execute SQL and store user
Spring Container	<code>setUserService()</code> or constructor	Internally via <code>@Autowired</code>	To inject dependency at runtime

Summary

- **The flow is: Client → Controller → Service → DAO → DB**
- **Each layer delegates the job down** to keep logic separated and maintain loose coupling.
- **Spring Container is responsible for calling the constructor or setter methods** to wire dependencies automatically (that's **IoC/DI**).

Day 14

Model 1 Architecture

Technologies Used:

- **Servlets** and **JSP**

How it Works:

- Client request is directly handled by **JSP** or **Servlet**.
- JSPs contain both **presentation logic** and **business logic**.
- Control/navigation logic is embedded within each page.

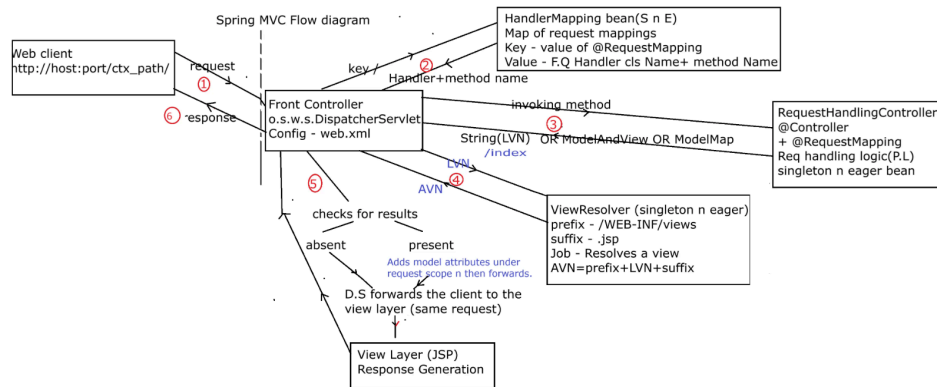
Advantages:

- Simple and quick to develop.
- Suitable for **small-scale applications**.

Disadvantages:

- **Poor separation of concerns** — mixing of HTML (presentation) with Java code (logic).

- If JSP/Servlet names change, you have to update **everywhere** — hard maintenance.
- No centralized control — each page decides where to go next.
- Difficult to **scale or extend** the application.



◆ Model 2 Architecture (MVC)

✓ Technologies Used:

- **Servlet (Controller) + JSP (View) + JavaBeans/POJOs (Model)**

⚙ How it Works:

- Client sends request → handled by a **Controller (Servlet)**.
- Controller interacts with **Model** (Java classes with business logic/data).
- Controller forwards results to **View** (JSP) for rendering output.

📌 MVC Breakdown:

Component	Role	Example
Model	Contains business logic & data	JavaBeans, POJOs
View	Presentation layer	JSP pages
Controller	Handles request/response & routing	Servlet/Filter

✓ Advantages:

- **Centralized control** — only controller decides page flow.
- Clean **separation of concerns** → easy to maintain, test, and extend.
- Better suited for **large-scale applications**.
- Easy to reuse code and logic (components are decoupled).

✗ Disadvantages:

- Initial development is **more complex**.
- Implementing a centralized controller manually can be tedious.
- That's why **Spring MVC** is widely adopted — it provides:
 - **Front Controller** (`DispatcherServlet`)
 - **Handler Mapping**
 - **View Resolver**
 - And many other reusable components.

✓ Summary Table:

Feature	Model 1	Model 2 (MVC)
Separation of Concerns	No	Yes
Navigation Control	Decentralized (per page)	Centralized (in controller)
Maintenance & Scalability	Hard	Easy
Best Suited For	Small projects	Medium to large-scale projects
Technologies Used	JSP, Servlet	JSP, Servlet, JavaBeans

◆ 3. Model 2 Architecture (MVC)

Flow:

```

Web Client
↓
Front Controller (Servlet)
↓
→ Forwards to appropriate Servlet for business logic
OR
→ JSP for response generation (View)
↓
Services → JavaBeans → DAO Layer → POJOs → DB
  
```

🔄 MVC Components:

Role	Responsibility
Controller	Handles requests & navigation (<code>Servlet</code>)
Model	Contains data/state & business logic (<code>Beans + DAO</code>)
View	Presents data to user (<code>JSP</code>)

✓ Advantages:

- Centralized control
- Clear separation of concerns
- Scalable and maintainable

✓ New Concepts Introduced:

1. Field-level Dependency Injection (DI) without Setters or Constructors

- **Annotation-based injection directly into private fields** using:

```
@Autowired
private Teacher myTeacher;
```

- Exception scenarios:
 - **No match** → `NoSuchBeanDefinitionException`
 - **Multiple matches** → `NoUniqueBeanDefinitionException`
 - **Exact match** → Works fine
- Using `@Autowired(required=false)` :
 - Avoids exception if bean not found, but may cause `NullPointerException` if used without null check
- Using `@Qualifier("beanName")` with `@Autowired` :
 - Resolves ambiguity by **injecting specific bean by name**

2. Front Controller Pattern

- Concept of a **centralized dispatcher** that intercepts all incoming requests
- Used in frameworks like **Spring MVC** (via `DispatcherServlet`) and **Struts2**
- Main responsibility: **navigation controller**

3. Spring MVC Internal Flow (Non-custom, framework-driven)

Step-by-step request flow explained:

- `DispatcherServlet` intercepts all requests
- Uses `@RequestMapping`, `@GetMapping`, `@PostMapping` to map methods
- **HandlerMapping** matches URL to controller methods
- Controller returns a **Logical View Name (LVN)**
- **ViewResolver** converts LVN to **Actual View Name (AVN)**

```
AVN = prefix + LVN + suffix (e.g., "/WEB-INF/views/" + "index" + ".jsp")
```

- `DispatcherServlet`:
 - Stores **model attributes** in request scope
 - Forwards to JSP (view layer)
 - Sends final response to client

4. `ModelAndView` Usage

- Used to pass both **data (model)** and **view name**

```
return new ModelAndView("index", "student", studentObject);
```

- Accessible in JSP using:

```
${requestScope.student}
```

XML Configuration vs Annotations in Spring

1. Declaring Beans (**bean id class**)

- XML:

```
<bean id="myBean" class="com.example.MyClass" />
```

- Annotations:

- **@Component** — Generic bean
- **@Controller** — Web layer controller
- **@Service** — Business layer
- **@Repository** — DAO layer
- **@RestController** — REST controller
- **@ControllerAdvice** / **@RestControllerAdvice** — Global exception handlers

2. Scope Attribute

- XML:

```
<bean scope="singleton|prototype|request|session" />
```

- Annotation:

- **@Scope("singleton")**, etc.

3. Lazy Initialization

- XML:

```
<bean lazy-init="true" />
```

- Annotation:

- **@Lazy(true)** at class level

4. Init Method

- XML:

```
<bean init-method="initMethodName" />
```

- **Annotation:**
 - `@PostConstruct` on method

5. Destroy Method

- **XML:**

```
<bean destroy-method="destroyMethodName" />
```

- **Annotation:**
 - `@PreDestroy` on method

6–9. Autowiring Dependencies

XML	Annotation
<code>autowire="byType"</code>	<code>@Autowired</code> on setter
<code>autowire="constructor"</code>	<code>@Autowired</code> on constructor
<code>autowire="byName"</code>	<code>@Autowired @Qualifier("beanId")</code>
Field Injection	<code>@Autowired</code> directly on field

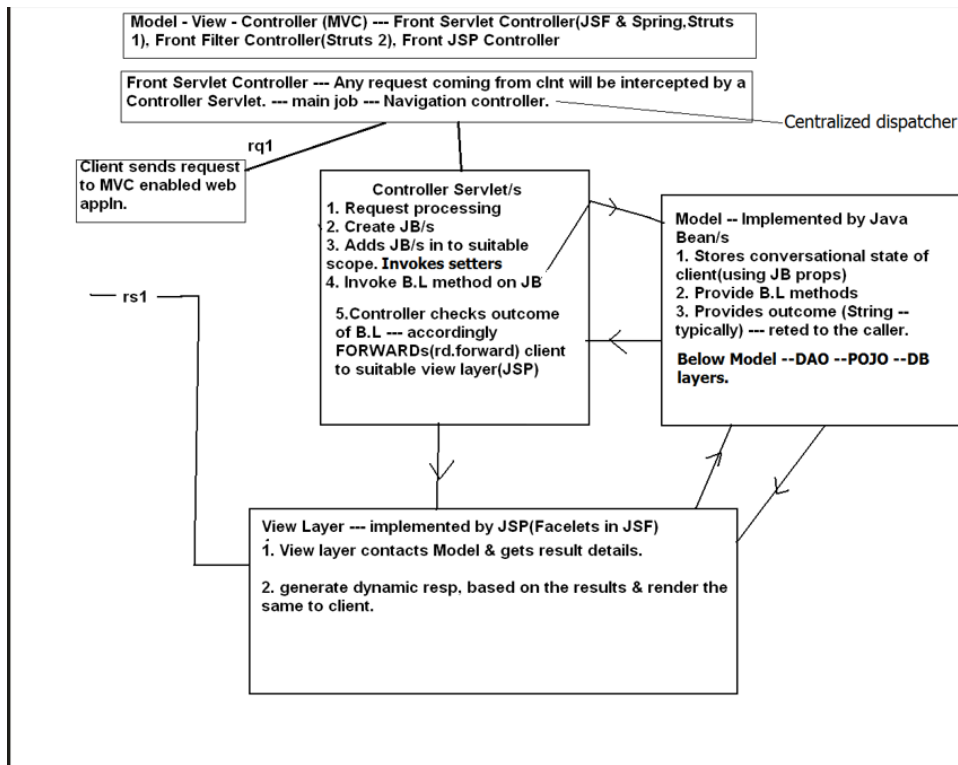
10–13. Configuration & Component Scanning

XML Tag	Annotation Equivalent
<code><bean></code>	<code>@Bean</code> method inside <code>@Configuration</code> class
<code><context:component-scan></code>	<code>@ComponentScan</code>
<code><name, value></code> property inject	<code>@Value("\${property}")</code> (SpEL support)
XML config file	<code>@Configuration</code> class

✓ Conclusion:

- **XML Configuration** is **declarative** and externalized.
- **Annotation-based Configuration** is **concise**, **less verbose**, and **closer to code**.
- **Hybrid approach** is also possible for flexibility.

MVC-2 Architecture



◆ Key Concepts Explained from the Diagram:

👤 1. Client Request (rq1)

- The client sends a request to a web application.
- **Front Servlet Controller** intercepts this request.
 - Example: `DispatcherServlet` in Spring, `ActionServlet` in Struts.

🕒 2. Controller Servlet (Central Dispatcher)

Handles:

1. **Request processing**
2. **Creating JavaBeans**
3. **Adding JavaBeans to proper scopes**
 - Request, session, application scope.
4. **Invoking setters** to populate beans.
5. **Calling Business Logic (B.L.)** methods on JavaBeans.
6. **Forwarding** the request to appropriate **View Layer** (like JSP) using `RequestDispatcher.forward()`.

🧠 3. Model (JavaBeans)

- Contains:
 1. **Client state** (stored using JavaBean properties).
 2. **Business Logic** methods.

3. **Returns outcome** (usually a `String`) that determines which view to load.

- **Below the Model:**

DAO → POJO → DB (data interaction)

4. View Layer (JSP / Facelets in JSF)

- **Fetches data from Model** using getters.
- **Generates dynamic response** (HTML) based on the model data.
- **Renders the result** back to the client (`rs1`).

Summary Table

Component	Responsibility	Example
Controller	Handles request, business logic flow, forwarding	<code>DispatcherServlet</code> (Spring)
Model	Stores state & business logic	JavaBeans, Services, DAO
View	Renders data for the client	JSP, Facelets (JSF)

MVC Advantages

1. Separation of Concerns

Division of responsibilities among various components: Model (data), View (UI), and Controller (logic).

2. Balanced Responsibility

No single component is overburdened with too many jobs.

3. Cleaner Architecture

Better separation between:

- Request processing
- Navigation
- Business logic
- Presentation logic

4. Reusability

Business logic components can be reused across different environments or platforms.

5. Independent Development

Each component (Model, View, Controller) can be implemented independently.

Example: You can change the UI (views) without touching the business logic.

6. Dynamic View Rendering

A single model can support multiple views, and the controller can dynamically decide which view to render based on logic.

Day 15

✓ What is a Model Attribute in Spring MVC?

◆ Definition:

A **Model Attribute** is a **name-value pair (String key, Object value)** used to **share data from a Controller (Handler) to the View layer (like JSP/Thymeleaf)** in a Spring MVC application.

◆ Purpose:

- To **pass business logic results** from the backend (Controller) to the frontend (View).
- Helps **populate UI elements** like tables, forms, dropdowns using server-side data.

✓ How Model Attributes Work Internally

1. The Controller creates model attributes (via `Model` , `ModelMap` , or `ModelAndView`)

2. DispatcherServlet (D.S) checks for any model attributes

3. If present:

- It stores them under the **request scope**
- Forwards the request to the view (JSP)

4. JSP accesses these using:

```
${requestScope.attrName}  
or  
${attrName} // since requestScope is default in JSP
```

✓ Ways to Add Model Attributes in Spring

◆ 1. Using `ModelAndView` (class)

```
@GetMapping("/home")  
public ModelAndView showHome() {  
    Student student = new Student("Akshay", 23);  
    return new ModelAndView("home", "student", student);  
}
```

- `"home"` : Logical View Name (LVN)
- `"student"` : model attribute key
- `student` : model attribute value

◆ 2. Using `Model` interface (recommended and simpler)

```
@GetMapping("/home")
public String showHome(Model model) {
    model.addAttribute("student", new Student("Akshay", 23));
    return "home"; // LVN
}
```

◆ 3. Using `ModelMap` (alternative to Model)

```
@GetMapping("/home")
public String showHome(ModelMap map) {
    map.addAttribute("student", new Student("Akshay", 23));
    return "home";
}
```

◆ 4. Using `@ModelAttribute` on Method Parameters

```
@PostMapping("/register")
public String registerStudent(@ModelAttribute Student student) {
    // Spring binds form fields to Student object
    return "success";
}
```

✓ Where Are Model Attributes Stored?

Scope	Who Stores?	When Cleared?
Request Scope	DispatcherServlet	After response is sent
Flash Scope	RedirectAttributes	After next request
Session Scope	HttpSession	Manually (or on logout)

✓ When and Where to Use Each

Scenario	Use
Passing data to JSP after forward	<code>Model</code> or <code>ModelAndView</code>
Redirect with a one-time message	<code>RedirectAttributes</code>
Storing user data after login	<code>HttpSession</code>

✓ JSP Usage Example (Thymeleaf/JSTL same idea)

Controller:

```
model.addAttribute("name", "Akshay");
```

JSP:

```
<p>Hello, ${name}</p>
```

✓ Internals Summary (Spring MVC Flow)

```
Client → D.S → Handler (Controller) → returns LVN + Model Attributes
    → D.S stores attrs in request scope
    → D.S forwards to View (e.g., home.jsp)
    → JSP renders data using ${}
```

Let me know if you'd like:

- A full **code + JSP** working example
- Comparison of `Model` vs `ModelMap` vs `ModelAndView`
- Difference between `@ModelAttribute` and `model.addAttribute`

✓ New Spring MVC Concepts Explained

1. What is a Model Attribute?

- A **model attribute** is a key-value pair (like `"user", userObject`) used to **pass data from the controller to the view layer**.
- Stored under **request scope** by default.
- Accessed in JSP using:

```
${requestScope.attrName}
```

✓ 2 Ways to Add Model Attributes in Handler Methods

1.1 Using `ModelAndView` class

- Combines both **model data** and **view name**:

```
return new ModelAndView("index", "student", studentObj);
```

- **Return type of handler:** `ModelAndView`

1.2 Using `Model` interface

- Spring auto-injects it via DI when added as method parameter:

```
public String handlerMethod(Model model) {
    model.addAttribute("key", value);
}
```

```
    return "viewName";  
}
```

- **Return type of handler:** `String` (just the Logical View Name)

✓ 3. Accessing Request Parameters in Controller

- Use `@RequestParam` :

```
public String handler(@RequestParam double price) {  
    // SC parses the parameter automatically  
}
```

- If the date format doesn't match Spring's default (`MM/dd/yyyy`), fix it using:

```
@RequestParam @DateTimeFormat(pattern = "yyyy-MM-dd") LocalDate manuDate
```

✓ 4. PRG Pattern (Post-Redirect-Get)

- Prevents duplicate form submission on page refresh.
- Use `redirect:` in return value:

```
return "redirect:/home";
```

Internals:

- Spring calls:

```
response.sendRedirect(...)
```

- Client re-issues a **GET request** to the redirected URL.

✓ 5. Session Scope in Spring

- Add data to session using DI:

```
public String login(HttpSession session) {  
    session.setAttribute("user", userObj);  
}
```

✓ 6. Flash Scope for One-Time Attributes

- Temporary attributes, available **only for the next request** (used in redirects).
- Add via:

```
public String postLogin(RedirectAttributes flash) {  
    flash.addFlashAttribute("status", "Login successful!");  
}
```

```
return "redirect:/dashboard";  
}
```

- Accessed in JSP with:

```
${requestScope.status}
```

✓ 7. Dynamic URLs with Spring Tags

- Avoid hardcoded URLs; use Spring's `<spring:url>` tag:

```
<%@ taglib prefix="spring" uri="http://www.springframework.org/tags" %>  
<spring:url var="loginUrl" value="/login"/>  
<form action="${loginUrl}" method="post">
```

- Ensures proper context path + `jsessionid` handling

✓ 8. Auto Redirect After Logout with Delay

- Set `Refresh` header to auto-navigate after a delay:

```
response.setHeader("refresh", "5;url=" + request.getContextPath());
```

- Redirects user after 5 seconds to your app's root context.

Spring MVC Request Flow

1. Client Request

A client (browser or frontend app) sends an HTTP request (GET, POST, etc.) to the Spring MVC application targeting a specific URL or endpoint.

2. DispatcherServlet (Front Controller)

- Acts as the central entry point for all requests in Spring MVC (MVC Model 2 pattern).
- Receives incoming requests and delegates them to appropriate components for processing.
- Configured via `web.xml` or auto-configured in Spring Boot.

3. Handler Mapping

- DispatcherServlet uses handler mappings to determine which controller method should handle the incoming request URL.

4. Controller (Request Handling Controller / Handler)

- The selected controller method executes business logic based on request data (parameters, path variables, etc.).
- Prepares the data model for the view.

5. Model and View Preparation

- Controller returns a model (data) and a logical view name.
- Model holds the data to display; view name indicates which template should be used.

6. View Resolution

- DispatcherServlet consults the configured View Resolver to map the logical view name to the actual view implementation (e.g., JSP, Thymeleaf).

7. View Rendering

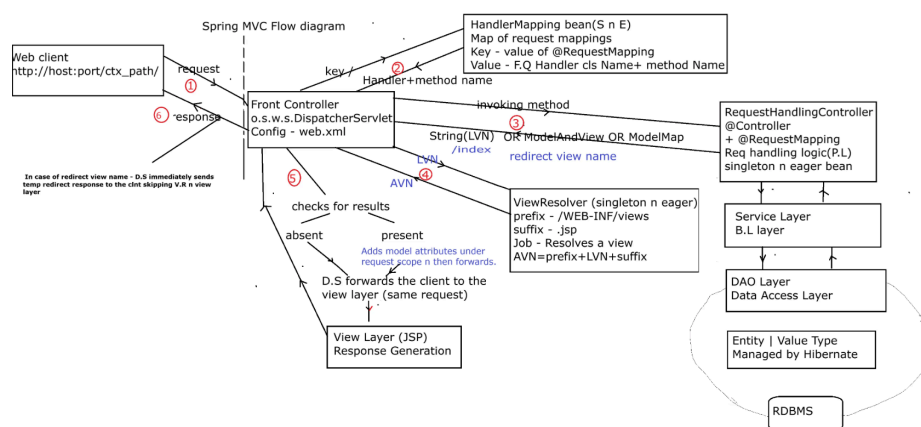
- The view engine (e.g., Thymeleaf, JSP) merges the model data with the template to generate the final HTML or other content.

8. Response Sent

- DispatcherServlet sends the rendered response back to the client over the network.

Additional Notes:

- Interceptors can intercept requests/responses before and after controller execution.
- Validators ensure data integrity during binding.
- Exception handlers handle errors gracefully within the MVC flow.
- Spring MVC offers rich features like flexible request mapping, data binding, form validation, and more.



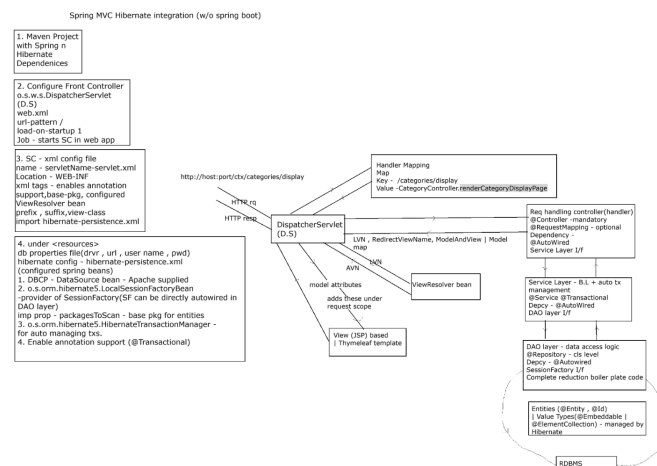
Breakdown of the Process:

1. **Web Client Requests:** A client sends a request to `http://host:port/ctx_path/`, initiating the process.
2. **Front Controller (DispatcherServlet):** Acts as a central controller, configured via `web.xml`, receiving all incoming requests.
3. **Handler Mapping:** Determines which controller should handle the request based on defined mappings.
4. **Controller (@Controller , @RequestMapping):** Processes the request, invoking appropriate service methods.
5. **Service Layer:** Contains business logic, coordinating data retrieval and manipulation.

6. **DAO (Data Access Object) Layer:** Handles interactions with the database using **Hibernate**.
7. **Entity Layer:** Represents objects mapped to database tables.
8. **RDBMS:** The relational database where data is stored and queried.
9. **View Resolver:** Determines the correct view (JSP file) to render the response.
10. **View Layer (JSP):** Generates the final response presented to the client.

This framework enables a structured, modular approach to web development by separating concerns—handling requests, processing logic, accessing data, and rendering views—all in distinct layers.

Do you want a real-world example of how this flow works in action?

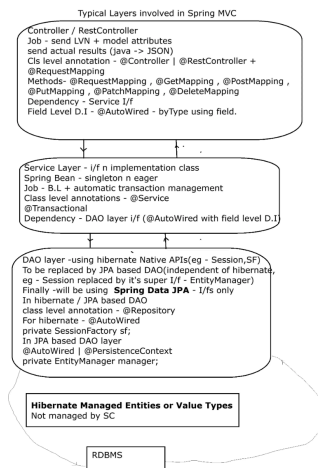


Breakdown of the Integration:

1. **Maven Dependencies:** The project includes necessary Spring and Hibernate dependencies.
2. **Front Controller (DispatcherServlet):** Defined in `web.xml`, handles incoming requests.
3. **Spring Configuration (servlet.xml):** Enables annotation support, sets up **ViewResolver**, and imports Hibernate configurations.
4. **Database Configuration:** Properties file defines database credentials and connection settings.
5. **Hibernate Configuration (hibernate-persistence.xml):**
 - Defines the **DataSource** bean.
 - Sets up **SessionFactory** for managing database sessions.
 - Configures **HibernateTransactionManager** for automatic transaction handling.
6. **Spring Layers:**
 - **Service Layer:** Contains business logic, manages transactions (`@Service`, `@Transactional`).
 - **DAO Layer:** Handles database interactions, reduces boilerplate code (`@Repository`, `@Autowired` SessionFactory).
7. **Entities:** Mapped using Hibernate annotations (`@Entity`, `@Id`, `@Embeddable`, `@ElementCollection`).

8. **View Resolver:** Determines the correct view (JSP or Thymeleaf templates) for rendering the response.

This setup allows developers to leverage Spring's MVC architecture with Hibernate for efficient database interactions without the need for Spring Boot.



Key Components in the Flow:

1. **Client Request:** The browser or web client sends an HTTP request to the application.
2. **Front Controller (`DispatcherServlet`):** Acts as the entry point, directing requests to the appropriate handler.
3. **Handler Mapping:** Determines which controller method will handle the request.
4. **Controller (`@Controller` , `@RequestMapping`):** Processes the request, invoking the service layer.
5. **Service Layer (`@Service`):** Contains business logic, ensuring smooth application operations.
6. **DAO Layer (`@Repository`):** Handles database operations using Hibernate.
7. **Hibernate (`SessionFactory` & `TransactionManager`):** Manages object persistence and transactions efficiently.
8. **Database (RDBMS):** Stores and retrieves requested data.
9. **View Resolver (`InternalResourceViewResolver`):** Identifies the appropriate JSP file to render.
10. **Response Generation:** The final processed data is sent back to the client as an HTTP response.

This structure ensures a **modular, scalable, and maintainable** web application by separating concerns effectively.

1. Integration of Spring MVC with Hibernate (without Spring Boot)

1.1 Spring MVC + Hibernate Setup (Non Spring Boot) — Key Points

- You configure **DispatcherServlet** in `web.xml` to handle all requests (Spring MVC entry point).
 - Spring's **Servlet Context (SC)** loads configuration XML (e.g., `spring-servlet.xml`).
 - You import your Hibernate configuration XML (`hibernate-persistence.xml`) inside your Spring config.
 - Beans configured in `hibernate-persistence.xml` include:
 - **DBCP (Database Connection Pool)**: Apache's BasicDataSource or similar.
 - **LocalSessionFactoryBean**: Creates Hibernate's `SessionFactory` , scans entity packages.
 - **HibernateTransactionManager**: Manages transactions declaratively.
 - Enable **annotation-driven transaction management** (`<tx:annotation-driven />`) to use `@Transactional` in service/DAO.
-

2. What happens internally when you deploy Spring MVC + Hibernate app?

- The web container (e.g., Tomcat) loads and initializes the **DispatcherServlet** (D.S).
 - D.S calls its `init()` method:
 - Starts the **Servlet Context (SC)** using the configured XML (usually named after servlet: `spring-servlet.xml`).
 - SC loads beans including your Controller, Service, DAO beans.
 - SC imports the `hibernate-persistence.xml` config that:
 - Sets up the database connection pool (DBCP).
 - Creates a `SessionFactory` bean for Hibernate sessions.
 - Configures transaction management.
 - SC scans base packages (via component-scan) and creates bean instances of Controllers, Services, DAOs, applying lifecycle management (singleton by default).
 - The app is now ready to handle HTTP requests routed through DispatcherServlet → Controller → Service → DAO → DB.
-

3. Details of `hibernate-persistence.xml` (Hibernate + Spring XML config)

- **Basic DataSource Bean**
 - Properties: JDBC driver, URL, username, password, connection pool initial and max sizes.
 - Provides connection pooling for DB connections (efficient DB usage).
- **LocalSessionFactoryBean**
 - Scans packages containing entity classes (`packagesToScan`).
 - Injects DataSource.
 - Provides Hibernate's `SessionFactory` for DAO layer.

- **HibernateTransactionManager**
 - Manages Hibernate transactions.
 - Depends on `SessionFactory`.
 - Enables declarative transaction handling via `@Transactional`.
 - Enable annotation-based transaction management:
`<tx:annotation-driven transaction-manager="transactionManager" />`
-

4. JSTL `<c:url>` for URL rewriting and context-relative URLs

- Example: `<c:url var="attrName" value="/path/to/resource"/>`
 - Generates a URL relative to the current context path, helps avoid hardcoding context root.
 - Ensures URLs work correctly regardless of deployment context (useful in JSPs).
-

5. Extensive XML Configuration Effort — Solution?

- Legacy Spring MVC + Hibernate requires lots of XML configuration (bean definitions, transactions, data sources, etc.).
 - Managing these XML files is tedious and error-prone.
 - **Spring Boot** is introduced to solve this by providing **auto-configuration**, reducing XML to near zero.
 - It has sensible defaults for DataSource, transaction management, Hibernate/JPA setup, embedded servlet containers.
 - Refer to your mentioned readmes for detailed steps and reasoning behind Spring Boot's inception.
-

6. Port existing Spring MVC + Hibernate app to Spring Boot

- You start by moving your existing web app into a Spring Boot project.
 - Spring Boot auto-configures DispatcherServlet, Hibernate/JPA, transaction managers, DataSource.
 - You can move from XML configs to Java-based config or `application.properties` / `application.yml`.
-

7. Common problem when porting native Hibernate apps to Spring Boot

- **Exception:**
`org.hibernate.HibernateException: No current session context configured`

- Occurs because Spring Boot prefers JPA over native Hibernate APIs.
 - Spring Boot does not support native `SessionFactory.getCurrentSession()` by default.
 - **Reason:**
Spring Boot uses JPA (`EntityManager`) as abstraction over native Hibernate APIs.
-

8. Solution: Replace native Hibernate APIs with JPA

- Use `EntityManager` (JPA API) instead of Hibernate `Session` .
 - Use `@PersistenceContext` to inject `EntityManager` in DAO.
 - `EntityManager` is thread-safe and integrates well with Spring Boot transaction management.
 - Benefit: Portability, cleaner API, better Spring support.
-

9. Common CRUD problems solved repeatedly in DAO layers?

- Yes, CRUD operations like save, update, delete, find are common in every DAO.
 - Writing these repeatedly is tedious and boilerplate-heavy.
-

10. Enter Spring Data JPA (Ultimate Data Access Abstraction)

- Spring Data JPA provides **out-of-the-box implementations** of common CRUD and query methods.
 - You define repository interfaces extending `JpaRepository<T, ID>` .
 - No need to write DAO implementations for simple CRUD operations.
 - Supports:
 - Pagination, sorting.
 - Custom queries with method names or JPQL.
 - Integration with Spring Boot with minimal config.
-

Useful Links for Spring Data JPA:

- API docs: <https://docs.spring.io/spring-data/data-jpa/docs/current/api/>
 - Reference docs: <https://docs.spring.io/spring-data/jpa/docs/current-SNAPSHOT/reference/html/#reference>
-

Summary

Stage	Description
Legacy Spring MVC + Hibernate	Lots of XML config, manual session and transaction management, native Hibernate APIs
Spring Boot + JPA	Auto-configuration, less XML, use JPA <code>EntityManager</code> instead of Hibernate <code>Session</code>
Spring Data JPA	Ready-made CRUD repo implementations, less boilerplate DAO code

If you want, I can help with:

- Sample configuration files for Spring MVC + Hibernate (non-Boot).
- Steps and code snippets for porting to Spring Boot + Spring Data JPA.
- Explanation or diagrams for Controller-Service-DAO-Entities flow.
- Example CRUD repo with Spring Data JPA.

What is JPA? (Java Persistence API)

JPA is a Java specification (not an implementation) for managing relational data in Java applications.

- It standardizes how Java objects (entities) are mapped to database tables.
- Provides an abstraction over various ORM (Object-Relational Mapping) frameworks like Hibernate, EclipseLink, OpenJPA.
- Defines interfaces such as `EntityManager`, `EntityTransaction`, and annotations like `@Entity`, `@Id`, `@OneToMany`, etc.
- Allows developers to write persistence logic without coupling directly to an ORM implementation.

Main purpose:

To **manage persistence** and **CRUD operations** in a consistent, vendor-neutral way.

Spring Data JPA — Simplifying DAO Layers

Spring Data JPA builds on top of JPA to **reduce boilerplate code** and speed up development by:

- Providing **ready-made implementations** of common data access patterns.
- Letting you **define repositories as interfaces only** — no need for DAO implementation classes.
- Generating query implementations automatically from method names.
- Supporting custom JPQL/native queries via annotations.
- Managing transactions declaratively.
- Translating low-level exceptions into Spring's consistent unchecked exceptions.

Key Concepts & Interfaces of Spring Data JPA

1. Repository Interfaces Hierarchy

Interface	Description	Main Methods
<code>Repository<T, ID></code>	Marker interface (empty), root of the hierarchy	—
<code>CrudRepository<T, ID></code>	Basic CRUD operations	<code>save()</code> , <code>findById()</code> , <code>findAll()</code> , <code>deleteById()</code> , <code>existsById()</code> , etc.
<code>PagingAndSortingRepository<T, ID></code>	Adds pagination and sorting	<code>findAll(Sort)</code> , <code>findAll(Pageable)</code>
<code>JpaRepository<T, ID></code>	Extends <code>PagingAndSortingRepository</code> ; adds JPA-specific methods	<code>flush()</code> , <code>saveAll()</code> , <code>deleteInBatch()</code> , <code>getOne()</code>

2. Usage Example

```
public interface ProductRepository extends JpaRepository<Product, Long> {  
    // No implementation needed here, CRUD methods are inherited  
}
```

You just declare the interface extending `JpaRepository`, and Spring Data will **auto-generate** the implementation.

3. CrudRepository — Common Methods

- `long count()` — number of entities
- `void delete(T entity)`
- `void deleteById(ID id)`
- `boolean existsById(ID id)`
- `Iterable<T> findAll()`
- `Optional<T> findById(ID id)`
- `<S extends T> S save(S entity)` — insert or update depending on ID presence

4. Spring Data JPA Advanced Features

A) Derived Query Methods (Query by Method Name)

You can define query methods by naming convention:

```
Optional<User> findByEmailAndPassword(String email, String password);  
List<User> findByDobBetween(LocalDate start, LocalDate end);  
List<Person> findByLastNameOrderByFirstnameAsc(String lastname);  
List<Person> findByAddressZipCode(String zipCode);
```

Spring Data parses the method name, generates the corresponding JPQL query automatically.

Supports:

- Property traversal (`findByAddressZipCode`)
- Logical operators (`And` , `Or`)
- Case ignoring (`IgnoreCase`)
- Result limiting (`First` , `Top`)
- Sorting and pagination

B) Custom Queries with @Query Annotation

You can provide JPQL or native SQL queries explicitly:

```
@Query("select u from User u where u.emailAddress = :em")
Optional<User> fetchUserByEmailAddress(@Param("em") String emailAddress);

@Modifying
@Query("update User u set u.firstname = ?1 where u.lastname = ?2")
int setFixedFirstnameFor(String firstname, String lastname);
```

- `@Modifying` annotation is used for update/delete queries.
- Supports both named parameters (`:param`) and positional parameters (`?1` , `?2`).

C) Transaction Management

- By default, Spring Data JPA uses `@Transactional` internally to manage transactions for repository methods.
- You can override transactional behavior per method or repository by using `@Transactional` annotations manually.
- Transactions are handled declaratively without boilerplate code in your DAO.

D) Exception Translation

- Spring Data repositories have the `@Repository` stereotype applied internally.
- This enables Spring's **exception translation mechanism**.
- Converts vendor-specific exceptions (e.g., Hibernate exceptions) into Spring's consistent `DataAccessException` hierarchy.
- You get consistent exception handling across different persistence technologies.

Summary: Benefits of Using Spring Data JPA

Benefit	Explanation
No DAO implementation classes	Just create interfaces extending <code>JpaRepository</code> ; Spring creates the implementation dynamically
Reduced boilerplate	CRUD, pagination, sorting, query derivation are automatic
Consistent transactions and exception handling	Built-in declarative transaction support and exception translation

Custom queries supported	Using method names or <code>@Query</code> annotations
Increased productivity	Focus on domain logic, not data access plumbing

Recommended Next Steps

- Define repository interfaces for your entities extending `JpaRepository<T, ID>`.
- Use method names or `@Query` annotations for custom queries.
- Inject repositories into services and call CRUD or custom methods directly.
- Remove all manual DAO implementations if migrating from native Hibernate DAO.

If you want, I can help with:

- Sample code snippets for creating repository interfaces.
- Examples of derived query methods and custom queries.
- How to configure Spring Data JPA in a Spring Boot app.
- Explaining transaction management and exception translation in more depth.

Day 16

What is a Web Service?

- **Web Service** = A solution for distributed computing.
- It's a **business functionality** (like banking, customer service, payment gateways, stock exchange info) exposed to remote clients over a **standard protocol** (mostly HTTP/HTTPS).
- Core idea: Enable **remote access to business logic** via the internet.
- Roles:
 - **Server** = service provider
 - **Client** = service consumer (accessor)

Why Web Services?

- To **export business logic** so clients can consume it remotely in a standardized, platform-independent way.
- Like **Java RMI** but:
 - RMI is **Java-only**, limiting interoperability.
 - Web services aim for **technology independence** and interoperability.

Alternatives & Evolution

1. CORBA (Common Object Request Broker Architecture)

- Platform-independent but complex and heavy.

- Requires **IDL (Interface Definition Language)** for object interfaces.

2. Web Services (W3C Standards from 2002)

- Introduced **WSDL (Web Service Definition Language)** and **SOAP** protocols.
- SOAP = XML-based messaging protocol running over HTTP.
- Standards aimed to create a universal way to describe and invoke services.

3. Java APIs for Web Services:

- Initially **JAX-RPC** (Java API for XML-based RPC)
- Now replaced by **JAX-WS** (Java API for XML Web Services), using SOAP.

4. SOAP Web Services

- Requires WSDL for describing the service.
- Uses UDDI for service discovery (rarely used today).
- Heavy XML payload → large bandwidth and complexity.

5. RESTful Web Services

- Became popular from 2004 onwards.
- Uses simple HTTP methods (GET, POST, PUT, DELETE).
- Supports multiple data formats (JSON, XML, etc.), typically JSON.
- Easier to build and consume compared to SOAP.

6. Java API for RESTful Services = JAX-RS

- Part of Java EE specs.
- Vendors: Apache, JBoss.
- Implementations: RESTeasy, Apache CXF.

Challenges & Spring's Role

- Even JAX-RS can be **complex to configure and set up**.
- **Spring Framework** simplifies development of both SOAP and RESTful services.
- Spring offers **Spring Web Services** and **Spring MVC REST** support with less boilerplate and easier integration.

Summary Table

Technology	Description	Protocol	Data Format	Ease of Setup	Interoperability
Java RMI	Java-specific remote calls	Java RMI	Java Object	Moderate	Java-only
CORBA	Cross-language remote calls	IIOP	Binary/XML	Complex	Cross-platform
SOAP + WSDL + UDDI	Standard XML web services	SOAP/HTTP	XML	Heavy, verbose	High

RESTful (JAX-RS)	Lightweight web services	HTTP	JSON/XML	Easier	High
Spring Web Services	Simplified SOAP/REST on Spring	SOAP/HTTP or HTTP	XML/JSON	Easy	High

What is REST?

REST = REpresentational State Transfer

- It's an **architectural style** for designing networked applications.
- REST is all about **transferring representations of resources** between client and server.
- Introduced by **Roy Fielding** in 2000 as part of his doctoral dissertation.

Key Concepts of REST

- **Resource:**

Anything that can be named or identified — e.g., Employee, Order, Flight, Cart, BlogPost, User, etc.

Each resource is identified by a **URI** (Uniform Resource Identifier).

- **Representation:**

The state or data of a resource, transferred as JSON, XML, HTML, or plain text. JSON is the most common and lightweight format today.

- **Client:**

The consumer or accessor of resources.

- **Server:**

The provider of resources.

RESTful Architecture

- **Resource-Oriented Architecture (ROA)** — REST revolves around resources, with a uniform interface to manipulate them.
- Uses **HTTP** as the underlying protocol.
- Makes use of standard HTTP methods to perform operations on resources.

Common HTTP Methods in REST

Method	Purpose
GET	Retrieve (read) a resource
POST	Create a new resource
PUT	Update (replace) a resource completely

PATCH	Partially update a resource
DELETE	Remove a resource

REST API (RESTful API)

- An **API** following REST principles, exposing endpoints for client-server communication.
- Allows different software systems to **communicate programmatically** over the internet.
- Used widely in business applications, e.g., payment gateways, weather data, social media, etc.
- Example:
 - OpenWeatherMap API (<https://openweathermap.org/current>) — fetches weather data.
 - Razorpay API (<https://razorpay.com/docs/api/basics/>) — payment gateway integration.

REST Architectural Constraints

To be strictly RESTful, a system must follow these constraints:

1. Uniform Interface:

Simplifies and decouples the architecture by having a standard way to interact with resources.

2. Stateless:

Each request from client to server must contain all information needed to understand and process it (no client context stored on server).

3. Cacheable:

Responses must explicitly define themselves as cacheable or not to improve performance.

4. Client-Server:

Separation of concerns — client and server operate independently.

5. Layered System:

Architecture can have multiple layers for scalability (e.g., load balancers, proxies).

6. Code on Demand (Optional):

Servers can send executable code to clients (like JavaScript), but this is optional.

Anatomy of a RESTful Web Service Request/Response

1. **URI:** Identifies the resource (<http://www.server.com/employee/1>).
2. **HTTP Verb:** Specifies operation (GET, POST, PUT, DELETE, PATCH).
3. **Request Headers:** Metadata like content type, authorization tokens, etc.
4. **Request Body:** Data sent by client (for POST/PUT/PATCH).
5. **Response Body:** Data sent by server (usually JSON or XML).

6. **Status Codes:** Indicate success or error (e.g., 200 OK, 404 Not Found, 500 Internal Server Error).
-

Why RESTful Web Services?

- Lightweight compared to SOAP.
 - Easy to use and understand.
 - Language and platform independent.
 - Widely adopted by modern web and mobile applications.
-

If you want, I can help you with:

- Sample REST API design
 - Implementing REST APIs with Java (Spring Boot) or other frameworks
 - Explanation of REST vs SOAP differences
 - Best practices for REST API design
-

REST architectural constraints

1. Client-Server Architecture

- **Separation of concerns:** Client and server are independent.
 - Clients request, servers respond without impacting each other's internal workings.
 - Enables different clients (mobile, web, IoT) to interact with the same server.
-

2. Statelessness

- Every request from client to server must contain **all information needed** to understand and process it.
 - Server doesn't store any client context between requests.
 - Each API call is independent, improving reliability and scalability.
-

3. Uniform Interface

- Standardized way for client-server communication.
 - Mostly built on **HTTP methods** (GET, POST, PUT, DELETE).
 - Common data formats like **JSON** ensure interoperability and simplicity.
 - This constraint decouples client and server, allowing independent evolution.
-

4. Layered System

- API architecture is divided into hierarchical layers.

- Layers only interact with adjacent layers, hiding complexity.
 - Supports scalability and enhances security by restricting direct communication.
-

5. Cacheability

- Responses explicitly indicate whether they are cacheable.
 - Clients (or intermediaries) can store responses temporarily.
 - Reduces unnecessary server calls and improves performance.
-

6. Code on Demand (Optional)

- Servers can send executable code (e.g., JavaScript) to clients.
 - Enhances client functionality dynamically.
 - Optional due to **security risks** and compatibility issues.
 - Mainly used in controlled environments, less common in public APIs.
-

Same Origin vs Cross Origin

- **Same Origin:**

A request is considered *same origin* if **all three** of these match between the requesting page (client) and the resource being requested (server):

1. **Domain** (e.g., `example.com`)
2. **Protocol** (e.g., `https`)
3. **Port** (e.g., `443` for HTTPS, `80` for HTTP)

Example:

- Request from `https://example.com:443/page` to `https://example.com:443/api/data` → **Same origin**

- **Cross Origin:**

If **any one** of domain, protocol, or port differs, the request is considered cross origin.

Example:

- Request from `https://example.com` to `http://example.com` (different protocol) → **Cross origin**
 - Request from `https://example.com` to `https://api.example.com` (different subdomain/domain) → **Cross origin**
 - Request from `https://example.com:443` to `https://example.com:8443` (different port) → **Cross origin**
-

How CORS works (Browser enforced)

1. When a webpage makes a **cross-origin HTTP request**, the browser automatically adds an `Origin` header to the request. This header specifies the origin (protocol + domain + port) of the calling site.
2. The cross-origin server receives this request and decides whether it allows access from that origin.
3. The server responds with an `Access-Control-Allow-Origin` header.

- If this header matches the origin sent in the `Origin` header (or is a wildcard), the browser allows the response to be read by the calling script.
- Otherwise, the browser blocks the response, and a CORS error appears in the console.

Example: Request and Response headers in CORS

Request headers sent by browser (Cross-origin request)

```
GET /data HTTP/1.1
Host: api.example.com
Origin: https://client.example.com
Accept: application/json
```

- `Origin` header tells the server the origin of the webpage making the request (`https://client.example.com`).

Response headers sent by server

- **Allowed origin:**

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: https://client.example.com
Content-Type: application/json
```

- The server explicitly allows the requesting origin.
- **If not allowed (or missing header):**

```
HTTP/1.1 200 OK
Content-Type: application/json
```

- No `Access-Control-Allow-Origin` header or mismatch → Browser blocks the response.

Summary

Step	Browser Request Header	Server Response Header	Effect
Browser sends cross-origin request	<code>Origin:</code> <code>https://client.example.com</code>		Tells server who is making the request
Server allows request from origin		<code>Access-Control-Allow-Origin:</code> <code>https://client.example.com</code>	Browser allows response
Server denies or omits allow header		(no or different <code>Access-Control-Allow-Origin</code>)	Browser blocks response, shows error

What is `ResponseEntity` ?

- `ResponseEntity<T>` is a **generic class** in Spring Framework that represents the **entire HTTP response** —

not just the body, but also the **status code** and **headers**.

- `T` is the type of the **response body** content.

Why use `ResponseEntity` instead of returning body directly?

- Allows full control over the HTTP response:
 - Set **status code** explicitly
 - Add **custom headers** if needed
 - Send the **body** content
- Improves clarity and flexibility in REST API design.
- Makes it easier to handle error responses with appropriate HTTP status.

Basic Constructors & Methods

- **Constructor:**

```
ResponseEntity(T body, HttpStatus status)
```

- **Builder style** (more readable and flexible):

```
ResponseEntity.status(HttpStatus.OK).body(bodyObject);
```

Example Usage in Controller

```
@GetMapping("/employee/{id}")
public ResponseEntity<Employee> getEmployee(@PathVariable Long id) {
    Employee emp = employeeService.findById(id);
    if (emp == null) {
        // Return 404 NOT FOUND with empty body
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(null);
    }
    // Return 200 OK with employee data
    return ResponseEntity.status(HttpStatus.OK).body(emp);
}
```

Adding headers example:

```
HttpHeaders headers = new HttpHeaders();
headers.add("Custom-Header", "value");
return new ResponseEntity<>(emp, headers, HttpStatus.OK);
```

Day 17

